

C++ 面试题

牛面 一站式学习与面试

2000+热门题库免费刷！题目全栈覆盖，助你拿offer!

海量题库刷不停，微信扫码立即开始!



趣味闯关

告别死记硬背，闯关练习陪你轻松进步，每一关都在悄悄变强。

1000+精品关卡等你来战

技术题库

春招、秋招最热题型全覆盖
多名大厂老鸟精心把关筛选

2k+道命中率超高必刷题

简历押题

大厂标准提取核心问题
问题-思路-答案三位一体

个性化精准预测

模拟面试

专项突破，垂直领域靶向强化
简历深挖，六大维度全面拷问

真实场景完整演练

牛面题库 让面试准备更高效 → niumianoffer.com

C++训练营：针对对面试求职的 C++ 开发特训，大厂导师全程带练，帮助学员从语法基础到项目实战，冲击高薪offer

大家好，我是小林。

说实话，C++ 的面试是所有语言里我觉得最难准备的，因为它考的东西实在太杂了。你以为把语法背熟就够了，结果面试官上来就问你虚函数表的内存布局、智能指针的引用计数实现，或者直接让你手写一个 `shared_ptr`。

这篇文章把 C++ 面试中高频出现的问题都整理下来了，基本上覆盖了从基础语法到内存管理、从面向对象到 STL、再到 C++11 新特性这一整条线。内容不算少，但我尽量把每个知识点都讲清楚，不只是给你一个结论，而是把背后的原理也说明白，因为很多面试官就是喜欢追着问“为什么”。

这里面有几块内容特别值得重点关注：

- **指针和内存管理**：这是 C++ 的核心，悬空指针、野指针、malloc/new 的区别、内存对齐，面试几乎必问。
- **虚函数和多态**：虚函数表的结构、纯虚函数、为什么析构函数要写成虚函数，这些是面向对象部分的重点。
- **STL 容器**：各种容器的底层实现、迭代器失效的场景，map 和 unordered_map 的区别，别只会用，要知道原理。
- **智能指针**：shared_ptr、unique_ptr、weak_ptr 的使用和循环引用问题，现在几乎是标配考点。
- **C++11 新特性**：右值引用、移动语义、lambda 表达式、auto 推导，这些在大厂面试里问得越来越多。

如果你是刚开始准备，建议先把 C++ 基础和面向对象的部分搞透，再去看 STL 和新特性。遇到不理解的地方多结合代码去看，光靠背是很难真正掌握的。

C++ 基础

什么是指针？指针的大小及用法？

指针：指向另外一种类型的复合类型。指针的大小：在 64 位计算机中，指针占 8 个字节空间。

```
#include<iostream>

using namespace std;

int main(){
    int *p = nullptr;
    cout << sizeof(p) << endl; // 8

    char *p1 = nullptr;
    cout << sizeof(p1) << endl; // 8
    return 0;
}
```

指针的用法：

指向普通对象的指针:

```
#include <iostream>

using namespace std;

class A
{
};

int main()
{
    A *p = new A();
    return 0;
}
```

指向常量对象的指针: 常量指针

```
#include <iostream>
using namespace std;

int main(void)
{
    const int c_var = 10;
    const int * p = &c_var;
    cout << *p << endl;
    return 0;
}
```

指向函数的指针: 函数指针

```
#include <iostream>
using namespace std;

int add(int a, int b){
    return a + b;
}

int main(void)
```

```

{
    int (*fun_p)(int, int);
    fun_p = add;
    cout << fun_p(1, 6) << endl;
    return 0;
}

```

指向对象成员的指针，包括指向对象成员函数的指针和指向对象成员变量的指针。特别注意：定义指向成员函数的指针时，要标明指针所属的类。

```

#include <iostream>
using namespace std;

class A
{
public:
    int var1, var2;
    int add(){
        return var1 + var2;
    }
};

int main()
{
    A ex;
    ex.var1 = 3;
    ex.var2 = 4;

    int *p = &ex.var1; // 指向对象成员变量
    cout << *p << endl;

    int (A::*fun_p)(); // 成员函数指针
    fun_p = &A::add;   // 必须加 &

    cout << (ex.*fun_p)() << endl; // 调用成员函数
    return 0;
}

```

this 指针：指向类的当前对象的指针常量。

```

#include <iostream>

```

```
#include <cstring>
using namespace std;

class A
{
public:
    void set_name(string tmp)
    {
        this->name = tmp;
    }
    void set_age(int tmp)
    {
        this->age = age;
    }
    void set_sex(int tmp)
    {
        this->sex = tmp;
    }
    void show()
    {
        cout << "Name: " << this->name << endl;
        cout << "Age: " << this->age << endl;
        cout << "Sex: " << this->sex << endl;
    }

private:
    string name;
    int age;
    int sex;
};

int main()
{
    A *p = new A();
    p->set_name("Alice");
    p->set_age(16);
    p->set_sex(1);
    p->show();

    return 0;
}
```

什么是野指针和悬空指针？

悬空指针：若指针指向一块内存空间，当这块内存空间被释放后，该指针依然指向这块内存空间，此时，称该指针为“悬空指针”。

```
void *p = malloc(size);
free(p);
// 此时，p 指向的内存空间已释放， p 就是悬空指针。
```

“野指针”是指不确定其指向的指针，未初始化的指针为“野指针”。

```
void *p;
// 此时 p 是“野指针”。
```

指针和引用的区别是什么？

- 指针所指向的内存空间在程序运行过程中可以改变，而引用所绑定的对象一旦绑定就不能改变。（是否可变）
- 指针本身在内存中占有内存空间，引用相当于变量的别名，在内存中不占内存空间。（是否占内存）
- 指针可以为空，但是引用必须绑定对象。（是否可为空）
- 指针可以有多级，但是引用只能一级。（是否能为多级）

常量指针和指针常量的区别是什么？

常量指针：常量指针本质上是个指针，只不过这个指针指向的对象是常量。也就是说，不可以通过对指针解引用修改指针指向的内容，而可以修改指针的指向。

特点：const 的位置在指针声明运算符 * 的左侧。只要 const 位于 * 的左侧，无论它在类型名的左边或右边，都表示指向常量的指针。（可以这样理解，* 左侧表示指针指向的对象，该对象为常量，那么该指针为常量指针。）

```
const int * p;
int const * p;
```

注意 1：指针指向的对象不能通过这个指针来修改，也就是说常量指针可以被赋值为变量的地址，之所以叫做常量指针，是限制了通过这个指针修改变量的值。

```
#include <iostream>
using namespace std;

int main()
{
    const int c_var = 8;
    const int *p = &c_var;
    *p = 6;           // error: assignment of read-only location '* p'
    return 0;
}
```

注意 2：虽然常量指针指向的对象不能变化，可是因为常量指针本身是一个变量，因此，可以被重新赋值。例如：

```
#include <iostream>
using namespace std;

int main()
{
    const int c_var1 = 8;
    const int c_var2 = 8;
    const int *p = &c_var1;
    p = &c_var2;
    return 0;
}
```

指针常量的本质上是常量，只不过这个常量的值是一个指针，和上面那个恰恰相反，不可以修改指针的指向，但可以通过对指针解引用修改指针指向的内容。

特点：const 位于指针声明操作符右侧，表明该对象本身是一个常量，*左侧表示该指针指向的类型，即以*为分界线，其左侧表示指针指向的类型，右侧表示指针本身的性质。

```
const int var;
int * const c_p = &var;
```

注意 1: 指针常量的值是指针, 这个值因为是常量, 所以指针本身不能改变。

```
#include <iostream>
using namespace std;

int main()
{
    int var, var1;
    int * const c_p = &var;
    c_p = &var1; // error: assignment of read-only variable 'c_p'
    return 0;
}
```

注意 2: 指针的内容可以改变。

```
#include <iostream>
using namespace std;

int main()
{
    int var = 3;
    int * const c_p = &var;
    *c_p = 12;
    return 0;
}
```

函数指针和指针函数的区别是什么?

指针函数本质是一个函数, 只不过该函数的返回值是一个指针。相对于普通函数而言, 只是返回值是指针。

```
#include <iostream>
using namespace std;

struct Type
{
    int var1;
    int var2;
};
```

```

Type * fun(int tmp1, int tmp2){
    Type * t = new Type();
    t->var1 = tmp1;
    t->var2 = tmp2;
    return t;
}

int main()
{
    Type *p = fun(5, 6);
    return 0;
}

```

函数指针本质是一个指针变量，只不过这个指针指向一个函数。函数指针即指向函数的指针。举例：

```

#include <iostream>
using namespace std;
int fun1(int tmp1, int tmp2)
{
    return tmp1 * tmp2;
}
int fun2(int tmp1, int tmp2)
{
    return tmp1 / tmp2;
}

int main()
{
    int (*fun)(int x, int y);
    fun = fun1;
    cout << fun(15, 5) << endl;
    fun = fun2;
    cout << fun(15, 5) << endl;
    return 0;
}
/*
运行结果：
75
3
*/

```

两者区别：

- 函数指针是个指针，专门指向函数的指针变量，比如能存某个函数的地址，通过它能调用对应的函数。
- 指针函数则是个函数，只是这个函数的返回值是指针类型，调用后会得到一个指针结果。

关键看名字最后俩字，“函数指针”核心是指针，用来指向函数；“指针函数”核心是函数，特点是返回指针。别被名字顺序迷惑，重点在结尾，一个是指针，一个是函数，这就是两者最根本的不同。

指针和引用值传递的区别是什么？

- 值传递是指将参数的值复制一份，传递给函数或方法进行操作。在值传递中，函数或方法对参数进行修改不会影响到原始的变量值。
- 指针引用是指将参数的内存地址传递给函数或方法，使得函数或方法可以直接访问和修改原始变量的值。在指针引用中，函数或方法对参数的修改会直接反映在原始变量上。

函数传参用引用的作用是什么？

可以避免避免拷贝，使用引用传参可以避免对大型对象进行复制。如果传递一个对象作为值参数，会触发对象的拷贝构造函数，造成额外的开销。而使用引用传参，可以直接在函数中操作原始对象，避免了拷贝操作。

参数传递时，值传递、引用传递、指针传递的区别？

参数传递的三种方式：

- 值传递：形参是实参的拷贝，函数对形参的所有操作不会影响实参。
- 指针传递：本质上是值传递，只不过拷贝的是指针的值，拷贝之后，实参和形参是不同的指针，通过指针可以间接的访问指针所指向的对象，从而可以修改它所指对象的值。
- 引用传递：当形参是引用类型时，我们说它对应的实参被引用传递。

```
#include <iostream>
using namespace std;

void fun1(int tmp){ // 值传递
```

```

        cout << &tmp << endl;
    }

    void fun2(int * tmp){ // 指针传递
        cout << tmp << endl;
    }

    void fun3(int &tmp){ // 引用传递
        cout << &tmp << endl;
    }

    int main()
    {
        int var = 5;
        cout << "var 在主函数中的地址: " << &var << endl;

        cout << "var 值传递时的地址: ";
        fun1(var);

        cout << "var 指针传递时的地址: ";
        fun2(&var);

        cout << "var 引用传递时的地址: ";
        fun3(var);
        return 0;
    }

    /*
    运行结果:
    var 在主函数中的地址: 0x23fe4c
    var 值传递时的地址: 0x23fe20
    var 指针传递时的地址: 0x23fe4c
    var 引用传递时的地址: 0x23fe4c
    */

```

说明：从上述代码的运行结果可以看出，只有在值传递时，形参和实参的地址不一样，在函数体内操作的不是变量本身。引用传递和指针传递，在函数体内操作的是变量本身。

C++全局变量、局部变量、静态全局变量、静态局部变量的区别？

C++ 变量根据定义的位置的不同的生命周期，具有不同的作用域，作用域可分为 6 种：全局作用域，局部作用域，语句作用域，类作用域，命名空间作用域和文件作用域。

从作用域看：

- 全局变量：具有全局作用域。全局变量只需在一个源文件中定义，就可以作用于所有的源文件。当然，其他不包含全局变量定义的源文件需要用extern 关键字再次声明这个全局变量。
- 静态全局变量：具有文件作用域。它与全局变量的区别在于如果程序包含多个文件的话，它作用于定义它的文件里，不能作用到其它文件里，即被static 关键字修饰过的变量具有文件作用域。这样即使两个不同的源文件都定义了相同名字的静态全局变量，它们也是不同的变量。
- 局部变量：具有局部作用域。它是自动对象（auto），在程序运行期间不是一直存在，而是只在函数执行期间存在，函数的一次调用执行结束后，变量被撤销，其所占用的内存也被收回。
- 静态局部变量：具有局部作用域。它只被初始化一次，自从第一次被初始化直到程序运行结束都一直存在，它和全局变量的区别在于全局变量对所有的函数都是可见的，而静态局部变量只对定义自己的函数体始终可见。

从分配内存空间看：

- 静态存储区：全局变量，静态局部变量，静态全局变量。
- 栈：局部变量。

全局变量定义在头文件中有什么问题？

如果在头文件中定义全局变量，当该头文件被多个文件 include 时，该头文件中的全局变量就会被定义多次，导致重复定义，因此不能再头文件中定义全局变量。

extern C 的作用是什么？

当 C++ 程序 需要调用 C 语言编写的函数，C++ 使用链接指示，即 extern "C" 指出任意非 C++ 函数所用的语言。举例：

```
// 可能出现在 C++ 头文件<cstring>中的链接指示
extern "C"{
    int strcmp(const char*, const char*);
}
```

sizeof(1==1) 在 C 和 C++ 中分别是什么结果？

C 语言代码：

```
#include<stdio.h>

void main(){
    printf("%d\n", sizeof(1==1));
}

/*
运行结果：
4
*/
```

C++ 代码：

```
#include <iostream>
using namespace std;

int main() {
    cout << sizeof(1==1) << endl;
    return 0;
}

/*
1
*/
```

C 和 C++ struct 的区别？

- 在 C 语言中 struct 是用户自定义数据类型；在 C++ 中 struct 是抽象数据类型，支持成员函数的定义。
- C 语言中 struct 没有访问权限的设置，是一些变量的集合体，不能定义成员函数；C++ 中 struct 可以和类一样，有访问权限，并可以定义成员函数。
- C 语言中 struct 定义的自定义数据类型，在定义该类型的变量时，需要加上 struct 关键字，例如：struct A var;，定义 A 类型的变量；而 C++ 中，不用加该关键字，例如：A var;

C++ 中 struct和Class区别是什么？

在 C++ 中， `struct` 和 `class` 是两种用于定义自定义数据类型的关键字，它们的核心功能相似（都能包含成员变量和成员函数），但存在一些关键区别，主要体现在[默认访问权限](#)和[默认继承方式](#)上：

- 默认访问权限不同： `struct` 默认访问权限为 `public` ， `class` 默认访问权限为 `private` 。比如：

```
struct A {
    int x; // 默认 public, 外部可直接访问
    void f() {} // 默认 public, 外部可调用
};

class B {
    int y; // 默认 private, 外部无法直接访问
    void g() {} // 默认 private, 外部无法直接调用
};

int main() {
    A a;
    a.x = 10; // 合法 (struct 成员默认 public)
    a.f();    // 合法

    B b;
    b.y = 20; // 编译错误 (class 成员默认 private)
    b.g();    // 编译错误
    return 0;
}
```

- 默认继承方式不同：当使用继承时，两者的默认继承权限也不同， `struct` 默认继承方式为 `public` 继承， `class` 默认继承方式为 `private` 继承。比如：

```
struct Base {
    int x;
};

// struct 默认 public 继承: Base 的 public 成员在 Derived1 中仍为 public
struct Derived1 : Base {
    // 可直接访问 Base::x, 外部也可通过 Derived1 对象访问 x
}
```

```
};

// class 默认 private 继承: Base 的 public 成员在 Derived2 中变为 private
class Derived2 : Base {
    // 可直接访问 Base::x, 但外部无法通过 Derived2 对象访问 x
};

int main() {
    Derived1 d1;
    d1.x = 10; // 合法 (public 继承)

    Derived2 d2;
    d2.x = 20; // 编译错误 (private 继承)
    return 0;
}
```

为什么有了 class 还保留 struct?

C++ 是在 C 语言的基础上发展起来的, 为了与 C 语言兼容, C++ 中保留了 struct。

struct 和 union 的区别是什么?

说明: union 是联合体, struct 是结构体。

区别:

- 联合体和结构体都是由若干个数据类型不同的数据成员组成。使用时, 联合体只有一个有效的成员; 而结构体所有的成员都有效。
- 对联合体的不同成员赋值, 将会对覆盖其他成员的值, 而对于结构体的对不同成员赋值时, 相互不影响。
- 联合体的大小为其内部所有变量的最大值, 按照最大类型的倍数进行分配大小; 结构体分配内存的大小遵循内存对齐原则。

```
#include <iostream>
using namespace std;

typedef union
{
    char c[10];
    char cc1; // char 1 字节, 按该类型的倍数分配大小
}
```

```

} u11;

typedef union
{
    char c[10];
    int i; // int 4 字节, 按该类型的倍数分配大小
} u22;

typedef union
{
    char c[10];
    double d; // double 8 字节, 按该类型的倍数分配大小
} u33;

typedef struct s1
{
    char c; // 1 字节
    double d; // 1 (char) + 7 (内存对齐) + 8 (double) = 16 字节
} s11;

typedef struct s2
{
    char c; // 1 字节
    char cc; // 1 (char) + 1 (char) = 2 字节
    double d; // 2 + 6 (内存对齐) + 8 (double) = 16 字节
} s22;

typedef struct s3
{
    char c; // 1 字节
    double d; // 1 (char) + 7 (内存对齐) + 8 (double) = 16 字节
    char cc; // 16 + 1 (char) + 7 (内存对齐) = 24 字节
} s33;

int main()
{
    cout << sizeof(u11) << endl; // 10
    cout << sizeof(u22) << endl; // 12
    cout << sizeof(u33) << endl; // 16
    cout << sizeof(s11) << endl; // 16
    cout << sizeof(s22) << endl; // 16
    cout << sizeof(s33) << endl; // 24

    cout << sizeof(int) << endl; // 4

```

```

    cout << sizeof(double) << endl; // 8
    return 0;
}

```

class 和 struct 的异同是什么？

- struct 和 class 都可以自定义数据类型，也支持继承操作。
- struct 中默认访问级别是 public，默认的继承级别也是 public；class 中默认访问级别是 private，默认的继承级别也是 private。
- 当 class 继承 struct 或者 struct 继承 class 时，默认的继承级别取决于 class 或 struct 本身，class（private 继承），struct（public 继承），即取决于派生类的默认继承级别。

```

struct A{};
class B : A{}; // private 继承
struct C : B{}; // public 继承

```

举例:

```

#include<iostream>

using namespace std;

class A{
public:
    void funA(){
        cout << "class A" << endl;
    }
};

struct B: A{ // 由于 B 是 struct, A 的默认继承级别为 public
public:
    void funB(){
        cout << "class B" << endl;
    }
};

class C: B{ // 由于 C 是 class, B 的默认继承级别为 private, 所以无法访问基类 B

```

```
};

int main(){
    A ex1;
    ex1.funA(); // class A

    B ex2;
    ex2.funA(); // class A
    ex2.funB(); // class B

    C ex3;
    ex3.funB(); // error: 'B' is not an accessible base of 'C'.
    return 0;
}
```

class 可以用于定义模板参数，struct 不能用于定义模板参数。

C 和 C++ static 的区别是什么？

在 C 语言中，使用 static 可以定义局部静态变量、外部静态变量、静态函数 在 C++ 中，使用 static 可以定义局部静态变量、外部静态变量、静态函数、静态成员变量和静态成员函数。因为 C++ 中有类的概念，静态成员变量、静态成员函数都是与类有关的概念。

C++ static 作用是什么？

static 定义静态变量，静态函数。

保持变量内容持久：static 作用于局部变量，改变了局部变量的生存周期，使得该变量存在于定义后直到程序运行结束的这段时间。

```
#include <iostream>
using namespace std;

int fun(){
    static int var = 1; // var 只在第一次进入这个函数的时初始化
    var += 1;
    return var;
}

int main()
```

```

{
    for(int i = 0; i < 10; ++i)
        cout << fun() << " "; // 2 3 4 5 6 7 8 9 10 11
    return 0;
}

```

隐藏：static作用于全局变量和函数，改变了全局变量和函数的作用域，使得全局变量和函数只能在定义它的文件中使用，在源文件中不具有全局可见性。（注：普通全局变量和函数具有全局可见性，即其他的源文件也可以使用。）

static 作用于类的成员变量和类的成员函数，使得类变量或者类成员函数和类有关，也就是说可以不定义类的对象就可以通过类访问这些静态成员。注意：类的静态成员函数中只能访问静态成员变量或者静态成员函数，不能将静态成员函数定义成虚函数。

```

#include<iostream>
using namespace std;

class A
{
private:
    int var;
    static int s_var; // 静态成员变量
public:
    void show()
    {
        cout << s_var++ << endl;
    }
    static void s_show()
    {
        cout << s_var << endl;
        // cout << var << endl; // error: invalid use of member 'A::a' i
        // show(); // error: cannot call member function 'void A::show(
    }
};

int A::s_var = 1; // 静态成员变量在类外进行初始化赋值，默认初始化为 0

int main()
{

    // cout << A::sa << endl; // error: 'int A::sa' is private within
    A ex;
    ex.show();
}

```

```
    A::s_show();  
}
```

static 在类中使用的注意事项有哪些？

static 静态成员变量：

- 静态成员变量是在类内进行声明，在类外进行定义和初始化，在类外进行定义和初始化的时候不要出现static关键字和private、public、protected 访问规则。
- 静态成员变量相当于类域中的全局变量，被类的所有对象所共享，包括派生类的对象。
- 静态成员变量可以作为成员函数的参数，而普通成员变量不可以。

```
#include <iostream>  
using namespace std;  
  
class A  
{  
public:  
    static int s_var;  
    int var;  
    void fun1(int i = s_var); // 正确，静态成员变量可以作为成员函数的参数  
    void fun2(int i = var);    // error: invalid use of non-static data  
};  
int main()  
{  
    return 0;  
}
```

静态数据成员的类型可以是所属类的类型，而普通数据成员的类型只能是该类类型的指针或引用。

```
#include <iostream>  
using namespace std;  
  
class A  
{  
public:  
    static A s_var; // 正确，静态数据成员  
    A var;          // error: field 'var' has incomplete type 'A'  
}
```

```
A *p;           // 正确，指针
A &var1;        // 正确，引用
};

int main()
{
    return 0;
}
```

static 静态成员函数：

- 静态成员函数不能调用非静态成员变量或者非静态成员函数，因为静态成员函数没有 this 指针。静态成员函数做为类作用域的全局函数。
- 静态成员函数不能声明成虚函数（virtual）、const 函数和 volatile 函数。

static 全局变量和普通全局变量的异同是什么？

相同点：

- 存储方式：普通全局变量和 static 全局变量都是静态存储方式。

不同点：

- 作用域：普通全局变量的作用域是整个源程序，当一个源程序由多个源文件组成时，普通全局变量在各个源文件中都是有效的；静态全局变量则限制了其作用域，即只在定义该变量的源文件内有效，在同一源程序的其它源文件中不能使用它。由于静态全局变量的作用域限于一个源文件内，只能为该源文件内的函数公用，因此可以避免在其他源文件中引起错误。
- 初始化：静态全局变量只初始化一次，防止在其他文件中使用。

C++ 静态变量的使用场景是什么？未初始化的全局静态变量呢？

静态变量（包括全局静态、局部静态、类静态成员）的核心特点是[生命周期贯穿程序运行始终，且作用域受限定](#)，常见使用场景如下：

全局静态变量（static 修饰的全局变量）

- 作用：限制变量仅在当前文件内可见（避免不同文件中同名变量冲突），但生命周期是

整个程序运行期间。

- 场景：当多个文件需要独立使用同名变量（如统计各模块的内部计数），但不希望被其他文件访问或修改时。例： `static int count = 0;`（仅当前 `.cpp` 文件可访问，其他文件即使声明 `extern int count` 也无法使用）。

局部静态变量（函数内的 `static` 变量）

- 作用：变量在函数第一次调用时初始化，后续调用不再重新初始化，值会被保留（生命周期全局，作用域仅限函数内）。
- 场景：记录函数被调用的次数（如 `static int call_count = 0;` `call_count++;`）；单例模式中，确保全局只存在一个实例（如函数内返回静态对象的指针）；避免频繁创建销毁临时对象（如工具函数中复用的缓冲区）。

类静态成员变量（`static` 修饰的类成员）

- 作用：属于整个类而非某个对象，所有对象共享该变量，生命周期全局，需在类外单独初始化。
- 场景：统计类的实例数量（如 `static int total;`，在构造函数中 `total++`，析构函数中 `total--`）；存储类级别的常量或共享配置（如 `static const int MAX_SIZE = 100;`）。

未初始化的全局静态变量

未初始化的全局静态变量（如 `static int a;`）有两个关键特性：

1. **自动初始化**：编译器会将其默认初始化为 **0**（包括数值类型为 0，指针类型为 `nullptr` 等）。这是因为全局静态变量存放在内存的 **BSS 段**（未初始化数据段），程序启动时系统会自动将该段所有数据清零。
2. **作用域限制**：和初始化的全局静态变量一样，仅在当前文件内可见，不影响其他文件的同名变量。

例：

```
// file1.cpp
static int uninit; // 未初始化，默认值为0，仅file1可见
```

java

```
// file2.cpp
static int uninit; // 与file1的uninit无关, 各自为0
```

未初始化的全局静态变量本质上是“带文件作用域的零初始化全局变量”，适合需要跨函数（但仅限当前文件）共享、且初始值为 0 的场景。

介绍const 作用及用法？

作用：

- const 修饰成员变量，定义成 const 常量，相较于宏常量，可进行类型检查，节省内存空间，提高了效率。
- const 修饰函数参数，使得传递过来的函数参数的值不能改变。
- const 修饰成员函数，使得成员函数不能修改任何类型的成员变量（mutable 修饰的变量除外），也不能调用非 const 成员函数，因为非 const 成员函数可能会修改成员变量。

在类中的用法：

const 成员变量：

- const 成员变量只能在类内声明、定义，在构造函数初始化列表中初始化。
- const 成员变量只在某个对象的生存周期内是常量，对于整个类而言却是可变的，因为类可以创建多个对象，不同类的 const 成员变量的值是不同的。因此不能在类的声明中初始化 const 成员变量，类的对象还没有创建，编译器不知道他的值。

const 成员函数：

- 不能修改成员变量的值，除非有 mutable 修饰；只能访问成员变量。
- 不能调用非常量成员函数，以防修改成员变量的值。

define 和 const 的区别是什么？

区别：

- 编译阶段：define 是在编译预处理阶段进行替换，const 是在编译阶段确定其值。
- 安全性：define 定义的宏常量没有数据类型，只是进行简单的替换，不会进行类型安全的检查；const 定义的常量是有类型的，是要进行判断的，可以避免一些低级的错误。

- 内存占用：define 定义的宏常量，在程序中使用多少次就会进行多少次替换，内存中有多个备份，占用的是代码段的空间；const 定义的常量占用静态存储区的空间，程序运行过程中只有一份。
- 调试：define 定义的宏常量不能调试，因为在预编译阶段就已经进行替换了；const 定义的常量可以进行调试。

const 的优点：

- 有数据类型，在定义式可进行安全性检查。
- 可调试。
- 占用较少的空间。

define 和 typedef 的区别是什么？

- 原理：#define 作为预处理指令，在编译预处理时进行替换操作，不作正确性检查，只有在编译已被展开的源程序时才会发现可能的错误并报错。typedef 是关键字，在编译时处理，有类型检查功能，用来给一个已经存在的类型一个别名，但不能在一个函数定义里面使用 typedef。
- 功能：typedef 用来定义类型的别名，方便使用。#define 不仅可以为类型取别名，还可以定义常量、变量、编译开关等。
- 作用域：#define 没有作用域的限制，只要是之前预定义过的宏，在以后的程序中都可以使用，而 typedef 有自己的作用域。
- 指针的操作：typedef 和 #define 在处理指针时不完全一样

```
#include <iostream>
#define INTPTR1 int *
typedef int * INTPTR2;

using namespace std;

int main()
{
    INTPTR1 p1, p2; // p1: int *; p2: int
    INTPTR2 p3, p4; // p3: int *; p4: int *

    int var = 1;
    const INTPTR1 p5 = &var; // 相当于 const int * p5; 常量指针，即不可以通过
    const INTPTR2 p6 = &var; // 相当于 int * const p6; 指针常量，不可使 p6 指
```

```
    return 0;  
}
```

volatile 的作用？是否具有原子性，对编译器有什么影响？

volatile 的作用：当对象的值可能在程序的控制或检测之外被改变时，应该将该对象声明为 volatile，告知编译器不应对这样的对象进行优化。

volatile不具有原子性。

volatile 对编译器的影响：使用该关键字后，编译器不会对相应的对象进行优化，即不会将变量从内存缓存到寄存器中，防止多个线程有可能使用内存中的变量，有可能使用寄存器中的变量，从而导致程序错误。

什么情况下一定要用 volatile，能否和 const 一起使用？

使用 volatile 关键字的场景：

- 当多个线程都会用到某一变量，并且该变量的值有可能发生改变时，需要用 volatile 关键字对该变量进行修饰；
- 中断服务程序中访问的变量或并行设备的硬件寄存器的变量，最好用 volatile 关键字修饰。

volatile 关键字和 const 关键字可以同时使用，某种类型可以既是 volatile 又是 const，同时具有二者的属性。

为什么一般将析构函数设置为虚函数？

析构函数被设为虚函数主要是为了解决基类指针指向派生类对象时的资源释放问题。

如果我们有一个基类指针，它实际上指向一个派生类对象，当我们删除这个基类指针时，如果析构函数不是虚函数，那么就只会调用基类的析构函数，而不会调用派生类的析构函数。这可能会导致派生类对象的一些资源没有被正确释放，从而引发内存泄漏等问题。

如果我们将析构函数设置为虚函数，那么在删除基类指针时，会首先调用派生类的析构函数，然后再调用基类的析构函数，从而确保所有的资源都能被正确释放。

析构函数为什么通常是会做成一个虚函数呢？

如果一个类有虚函数，就应该为其定义一个虚析构函数。这是因为在使用delete操作符释放一个指向派生类对象的基类指针时，如果基类的析构函数不是虚函数，那么只会调用基类的析构函数，而不会调用派生类的析构函数，这样就会导致内存泄漏和未定义行为的问题。通过将析构函数定义为虚函数，可以确保在释放派生类对象时，先调用派生类的析构函数，再调用基类的析构函数，从而避免内存泄漏和未定义行为的问题。

为什么析构函数一般写为虚函数？

如果析构函数不被声明成虚函数，[则编译器实施静态绑定](#)，在删除基类指针时，只会调用基类的析构函数而不调用派生类析构函数，这样就会造成派生类对象析构不完全，造成内存泄漏。

所以在实现多态时，当用基类操作派生类，在析构时防止只析构基类而不析构派生类的状况发生，要将基类的析构函数声明为虚函数。

为什么构造函数不写为虚函数？

从存储空间角度：虚函数对应一个vtable，可是这个vtable其实是存储在对象的内存空间的。问题出来了，如果构造函数是虚的，就需要通过vtable来调用，可是对象还没有实例化，也就是内存空间还没有，无法找到vtable，所以构造函数不能是虚函数。

从使用角度：虚函数的作用在于通过父类的指针或者引用来调用它的时候能够变成调用子类的那个成员函数。而构造函数是在创建对象时自动调用的，不可能通过父类的指针或者引用去调用，因此也就规定构造函数不能是虚函数。

什么是内联函数？

在C++中，使用关键字"inline"可以声明一个内联函数。声明为内联函数的函数会在编译时被视为候选项，编译器会尝试将其展开，将函数体直接插入到调用点处。这样可以避免函数调用的开销，减少了函数调用的栈帧等额外开销，从而提高程序的执行效率。

宏定义（define）和内联函数（inline）的区别是什么？

- 内联函数是在编译时展开，而宏在编译预处理时展开；在编译的时候，内联函数直接被嵌入到目标代码中去，而宏只是一个简单的文本替换。

- 内联函数是真正的函数，和普通函数调用的方法一样，在调用点处直接展开，避免了函数的参数压栈操作，减少了调用的开销。而宏定义编写较为复杂，常需要增加一些括号来避免歧义。
- 宏定义只进行文本替换，不会对参数的类型、语句能否正常编译等进行检查。而内联函数是真正的函数，会对参数的类型、函数体内的语句编写是否正确等进行检查。

```
#include <iostream>

#define MAX(a, b) ((a) > (b) ? (a) : (b))
```

```
using namespace std;
```

```
inline int fun_max(int a, int b)
{
    return a > b ? a : b;
}
```

```
int main()
{
    int var = 1;
    cout << MAX(var, 5) << endl;
    cout << fun_max(var, 0) << endl;
    return 0;
}
```

```
/*
```

```
程序运行结果：
```

```
5
```

```
1
```

```
*/
```

内联函数有什么缺点？

内联函数的缺点主要有以下几点：

- 代码膨胀：内联函数会在每个调用它的地方进行代码替换，这可能导致代码膨胀。如果内联函数体非常大或者被频繁调用，会增加可执行文件的大小，可能导致缓存不命中，影响性能。
- 编译时间增加：内联函数需要在每个调用点进行代码替换，这会增加编译时间。特别是当内联函数被广泛使用时，编译时间可能会显著增加。

- 可读性降低：内联函数会将函数体嵌入到调用点，可能导致代码的可读性降低。函数体被分散在多个地方，可能会使代码难以理解和维护。

include " " 和 <> 的区别是什么？

include<文件名> 和 #include"文件名" 的区别：

- 查找文件的位置：include<文件名>在标准库头文件所在的目录中查找，如果没有，再到当前源文件所在目录下查找；#include"文件名" 在当前源文件所在目录中进行查找，如果没有；再到系统目录中查找。
- 使用习惯：对于标准库中的头文件常用 include<文件名>，对于自己定义的头文件，常用 #include"文件名"

void*是什么？

void* 是一种通用的指针类型，被称为"无类型指针"。它可以用来表示指向任何类型的指针，因为 void* 指针没有指定特定的数据类型。

由于 void* 是无类型的，它不能直接进行解引用操作，也不能进行指针运算。在使用 void* 指针时，需要将其转换为具体的指针类型才能进行操作。

void* 指针常用于需要在不同类型之间进行通用操作的情况，例如在函数中传递任意类型的指针参数或在动态内存分配中使用。

malloc的参数列表 void*怎么转化为int*的？

可以使用类型转换将 void* 指针转化为 int* 指针。以下是将 void* 指针转化为 int* 指针的示例代码：

```
void* voidPtr = malloc(sizeof(int)); // 分配内存并返回void*指针
int* intPtr = (int*)voidPtr;        // 将void*指针转化为int*指针

// 现在可以通过intPtr指针访问int类型的数据
*intPtr = 42;
```

在上述示例中，使用 malloc 函数分配了存储一个 int 类型数据所需的内存，并返回了一个 void* 指针。然后，通过将 void* 指针转换为 int* 指针，将其赋值给 intPtr 变

量。现在，可以通过 `intPtr` 指针访问和操作 `int` 类型的数据。

sizeof 和 strlen 的区别是什么？

`strlen` 是头文件中的函数，`sizeof` 是 C++ 中的运算符。`strlen` 测量的是字符串的实际长度（其源代码如下），以 `\0` 结束。而 `sizeof` 测量的是字符数组的分配大小。

`strlen` 源代码：

```
size_t strlen(const char *str) {
    size_t length = 0;
    while (*str++)
        ++length;
    return length;
}
```

```
#include <iostream>
#include <cstring>

using namespace std;

int main()
{
    char arr[10] = "hello";
    cout << strlen(arr) << endl; // 5
    cout << sizeof(arr) << endl; // 10
    return 0;
}
```

若字符数组 `arr` 作为函数的形参，`sizeof(arr)` 中 `arr` 被当作字符指针来处理，`strlen(arr)` 中 `arr` 依然是字符数组，从下述程序的运行结果中就可以看出。

```
#include <iostream>
#include <cstring>

using namespace std;

void size_of(char arr[])
{
    cout << sizeof(arr) << endl; // warning: 'sizeof' on array function
```

```

        cout << strlen(arr) << endl;
    }

    int main()
    {
        char arr[20] = "hello";
        sizeof(arr);
        return 0;
    }
    /*
    输出结果：
    8
    5
    */

```

- strlen 本身是库函数，因此在程序运行过程中，计算长度；而 sizeof 在编译时，计算长度；
- sizeof 的参数可以是类型，也可以是变量；strlen 的参数必须是 char* 类型的变量。

explicit 的作用是什么？

作用：用来声明类构造函数是显示调用的，而非隐式调用，可以阻止调用构造函数时进行隐式转换。只可用于修饰单参构造函数，因为无参构造函数和多参构造函数本身就是显示调用的，再加上 explicit 关键字也没有什么意义。

隐式转换：

```

#include <iostream>
using namespace std;

class A {
public:
    int var;
    A(int tmp) {
        var = tmp;
    }
};

int main() {
    A ex = 10; // 发生了隐式转换
    return 0;
}

```

```
}
```

上述代码中，`A ex = 10;` 在编译时，进行了隐式转换，将 10 转换成 A 类型的对象，然后将该对象赋值给 `ex`，等同于如下操作：

为了避免隐式转换，可用 `explicit` 关键字进行声明：

```
#include <iostream>
using namespace std;

class A {
public:
    int var;
    explicit A(int tmp) {
        var = tmp;
        cout << var << endl;
    }
};

int main() {
    A ex(100);
    A ex1 = 10; // error: conversion from 'int' to non-scalar type 'A' r
    return 0;
}
```

memcpy 函数的底层原理是什么？

`memcpy` 函数的底层原理简单说就是直接操作内存块的二进制数据。它会从源地址开始，逐个字节（或按更高效的块）复制数据到目标地址，直到复制完指定的字节数。

底层实现通常会做优化，比如对对齐的内存块用更大的单位（如 4 字节、8 字节）批量复制，比单字节循环更快；对未对齐的部分先用单字节处理到对齐位置，再用块复制。整个过程不关心数据类型，纯粹按字节搬运，所以复制后目标内存和源内存的二进制内容完全一致，但不会处理像字符串结束符这类特殊情况。

```
void *memcpy(void *dst, const void *src, size_t size)
{
    char *psrc;
    char *pdst;
```

```

if (NULL == dst || NULL == src)
{
    return NULL;
}

if ((src < dst) && (char *)src + size > (char *)dst) // 出现地址重叠的情况
{
    psrc = (char *)src + size - 1;
    pdst = (char *)dst + size - 1;
    while (size--)
    {
        *pdst-- = *psrc--;
    }
}
else
{
    psrc = (char *)src;
    pdst = (char *)dst;
    while (size--)
    {
        *pdst++ = *psrc++;
    }
}

return dst;
}

```

strcpy 函数有什么缺陷？

strcpy 函数的缺陷：strcpy 函数不检查目的缓冲区的大小边界，而是将源字符串逐一的全部赋值给目的字符串地址起始的一块连续的内存空间，同时加上字符串终止符，会导致其他变量被覆盖。

```

#include <iostream>
#include <cstring>
using namespace std;

int main()
{
    int var = 0x11112222;

```

```

char arr[10];
cout << "Address : var " << &var << endl;
cout << "Address : arr " << &arr << endl;
strcpy(arr, "hello world!");
cout << "var:" << hex << var << endl; // 将变量 var 以 16 进制输出
cout << "arr:" << arr << endl;
return 0;
}

/*
Address : var 0x23fe4c
Address : arr 0x23fe42
var:11002164
arr:hello world!
*/

```

- 说明：从上述代码中可以看出，变量 var 的后六位被字符串 “hello world!” 的 “d!\0” 这三个字符改变，这三个字符对应的 ascii 码的十六进制为：\0(0x00)，!(0x21)，d(0x64)。
- 原因：变量 arr 只分配的 10 个内存空间，通过上述程序中的地址可以看出 arr 和 var 在内存中是连续存放的，但是在调用 strcpy 函数进行拷贝时，源字符串 “hello world!” 所占的内存空间为 13，因此在拷贝的过程中会占用 var 的内存空间，导致 var 的后六位被覆盖。

C++编译

C++的编译过程介绍一下？

C++的编译过程经过了预处理、编译、汇编和链接四个主要阶段：



- 预处理：预处理阶段会对源代码进行处理，主要包括展开宏定义、处理条件编译指令（如#include、#define、#ifdef等）以及删除注释等。预处理的结果是生成一个经过宏展开和条件处理后的纯C++源代码文件。

- 编译（Compilation）：编译阶段将预处理后的源代码翻译为汇编语言，生成汇编代码。编译器会进行词法分析、语法分析和语义分析，检查代码的正确性，并生成中间代码表示。
- 汇编：汇编阶段将汇编代码转换为机器可以执行的目标文件。汇编器会将汇编代码转化为机器指令，并生成与机器硬件平台相关的目标文件（通常以".obj"或".o"为扩展名）。
- 链接：链接阶段将目标文件与其他必要的库文件链接在一起，生成可执行程序。链接器会解析目标文件中的符号引用，将其与其他目标文件或库文件中的符号定义进行匹配，最终生成一个完整的可执行文件。在链接阶段，还会进行地址重定位、符号解析、符号表生成等操作，确保程序的正确执行。

静态链接库和动态链接库有什么区别？

- 链接方式：静态链接库在编译链接时会被完整地复制到可执行文件中，成为可执行文件的一部分；而动态链接库在编译链接时只会在可执行文件中包含对库的引用，实际的库文件在运行时由操作系统动态加载。
- 文件大小：静态链接库会使得可执行文件的大小增加，因为库的代码被完整地复制到可执行文件中；而动态链接库不会增加可执行文件的大小，因为库的代码在运行时才会被加载。
- 内存占用：静态链接库在运行时会被完整地加载到内存中，占用固定的内存空间；而动态链接库在运行时才会被加载，可以在多个进程之间共享，减少内存占用。
- 可扩展性：动态链接库的可扩展性更好，可以在不修改可执行文件的情况下替换或添加新的库文件，而静态链接库需要重新编译链接。

C++面向对象

什么是面向对象？面向对象的三大特性

面向对象：对象是指具体的某一个事物，这些事物的抽象就是类，类中包含数据（成员变量）和动作（成员方法）。

面向对象的三大特性：

- 封装：将具体的实现过程和数据封装成一个函数，只能通过接口进行访问，降低耦合性。
- 继承：子类继承父类的特征和行为，子类有父类的非 private 方法或成员变量，子类可以对父类的方法进行重写，增强了类之间的耦合性，但是当父类中的成员变量、成员函

数或者类本身被 final 关键字修饰时，修饰的类不能继承，修饰的成员不能重写或修改。

- 多态：多态就是不同继承类的对象，对同一消息做出不同的响应，基类的指针指向或绑定到派生类的对象，使得基类指针呈现不同的表现方式。

C++特性介绍一下？

- 封装是将一些数据和函数封装到类中，这样外层调用类只会调用到设计者想让他调用的方法；
- 继承的话，我常是设计一个基类，然后分别设置子类去继承基类的一些方法，尤其是虚函数，针对不同子类的特点对虚函数进行重写。
- 继承还有公有和私有两种方法，公有继承是将基类的成员都原封不动的继承下来，私有继承则会将其改为私有部分；多态的话，是有函数重载和之前提到的虚函数，函数重载是可以使得相同的函数面对不同的参数个数或者类型进行不同的方式实现。

如何理解 C++ 是面向对象编程？

说明：该问题最好结合自己的项目经历进行展开解释，或举一些恰当的例子，同时对比下面面向过程编程。

- 面向过程编程：一种以执行程序操作的过程或函数为中心编写软件的方法。程序的数据通常存储在变量中，与这些过程是分开的。所以必须将变量传递给需要使用它们的函数。缺点：随着程序变得越来越复杂，程序数据与运行代码的分离可能会导致问题。例如，程序的规范经常会发生变化，从而需要更改数据的格式或数据结构的设计。当数据结构发生变化时，对数据进行操作的代码也必须更改为接受新的格式。查找需要更改的所有代码会为程序员带来额外的工作，并增加了使代码出现错误的机会。
- 面向对象编程（Object-Oriented Programming, OOP）：以创建和使用对象为中心。一个对象（Object）就是一个软件实体，它将数据和程序在一个单元中组合起来。对象的数据项，也称为其属性，存储在成员变量中。对象执行的过程被称为其成员函数。将对象的数据和过程绑定在一起则被称为封装。

面向对象编程进一步说明：

- 面向对象编程将数据成员和成员函数封装到一个类中，并声明数据成员和成员函数的访问级别（public、private、protected），以便控制类对象对数据成员和函数的访问，对数据成员起到一定的保护作用。而且在类的对象调用成员函数时，只需知道成员函数的名、参数列表以及返回值类型即可，无需了解其函数的实现原理。当类内部的数据成员或者成员函数发生改变时，不影响类外部的代码。

重载、重写、隐藏的区别是什么？

重载：是指同一可访问区内被声明几个具有不同参数列（参数的类型、个数、顺序）的同名函数，根据参数列表确定调用哪个函数，重载不关心函数返回类型。

```
class A {
public:
    void fun(int tmp);
    void fun(float tmp);           // 重载 参数类型不同（相对于上一个函数）
    void fun(int tmp, float tmp1); // 重载 参数个数不同（相对于上一个函数）
    void fun(float tmp, int tmp1); // 重载 参数顺序不同（相对于上一个函数）
    int fun(int tmp);              // error: 'int A::fun(int)' cannot be o
};
```

隐藏：是指派生类的函数屏蔽了与其同名的基类函数，主要只要同名函数，不管参数列表是否相同，基类函数都会被隐藏。

```
#include <iostream>
using namespace std;

class Base {
public:
    void fun(int tmp, float tmp1) {
        cout << "Base::fun(int tmp, float tmp1)" << endl;
    }
};

class Derive : public Base {
public:
    void fun(int tmp) {
        cout << "Derive::fun(int tmp)" << endl;
    } // 隐藏基类中的同名函数
};

int main() {
    Derive ex;
    ex.fun(1);           // Derive::fun(int tmp)
    ex.fun(1, 0.01);     // error: candidate expects 1 argument, 2 provided
    return 0;
}
```

说明：上述代码中 `ex.fun(1, 0.01);` 出现错误，说明派生类中将基类的同名函数隐藏了。若是想调用基类中的同名函数，可以加上类型名指明 `ex.Base::fun(1, 0.01);`，这样就可以调用基类中的同名函数。

重写(覆盖)：是指派生类中存在重新定义的函数。函数名、参数列表、返回值类型都必须同基类中被重写的函数一致，只有函数体不同。派生类调用时会调用派生类的重写函数，不会调用被重写函数。重写的基类中被重写的函数必须有 `virtual` 修饰。

```
#include <iostream>
using namespace std;

class Base {
public:
    virtual void fun(int tmp) {
        cout << "Base::fun(int tmp) : " << tmp << endl;
    }
};

class Derived : public Base {
public:
    virtual void fun(int tmp) {
        cout << "Derived::fun(int tmp) : " << tmp << endl;
    } // 重写基类中的 fun 函数
};

int main() {
    Base *p = new Derived();
    p->fun(3); // Derived::fun(int) : 3
    return 0;
}
```

重写和重载的区别：

- 范围区别：对于类中函数的重载或者重写而言，重载发生在同一个类的内部，重写发生在不同的类之间（子类和父类之间）。
- 参数区别：重载的函数需要与原函数有相同的函数名、不同的参数列表，不关注函数的返回值类型；重写的函数的函数名、参数列表和返回值类型都需要和原函数相同，父类中被重写的函数需要有 `virtual` 修饰。
- `virtual` 关键字：重写的函数基类中必须有 `virtual` 关键字的修饰，重载的函数可以有

virtual 关键字的修饰也可以没有。

隐藏和重写，重载的区别：

- 范围区别：隐藏与重载范围不同，隐藏发生在不同类中。
- 参数区别：隐藏函数和被隐藏函数参数列表可以相同，也可以不同，但函数名一定相同；当参数不同时，无论基类中的函数是否被 virtual 修饰，基类函数都是被隐藏，而不是重写。

C++的多态是什么？怎么通过虚函数实现？

C++中的多态性指的是同一个操作作用于不同的对象时，可以产生不同的行为。多态性主要通过虚函数实现，能够让你以父类的指针或引用调用子类的实现，从而在运行时决定使用哪个函数。

C++中的多态通常分为两种主要类型：

- **编译时多态（静态多态）**：通过函数重载和运算符重载实现，在编译时确定调用哪个函数。
- **运行时多态（动态多态）**：通过虚函数实现，在运行时根据对象的实际类型决定调用的函数。实现原理是，每个包含虚函数的类都有一个虚函数表，这个表记录了该类对象可以调用的虚函数指针，每个对象也有一个隐式的虚函数指针，指向其类的虚函数表。在运行时，调用虚函数时，程序通过虚函数指针查找正确的函数。

虚函数是通过在基类中声明一个函数为 virtual 来实现的。这标志着这个函数可以被派生类重写（override）。当通过基类指针或引用调用虚函数时，C++会查找实际对象的类型，调用对应的子类实现，而不是基类的实现。

下面是一个简单的示例，展示了如何使用虚函数实现多态。

1. 定义基类和派生类

```
#include <iostream>
using namespace std;

// 基类
class Shape {
public:
    // 虚函数
    virtual void draw() {
```

```

        cout << "Drawing Shape" << endl;
    }
};

// 派生类: Circle
class Circle : public Shape {
public:
    void draw() override { // 重写虚函数
        cout << "Drawing Circle" << endl;
    }
};

// 派生类: Square
class Square : public Shape {
public:
    void draw() override { // 重写虚函数
        cout << "Drawing Square" << endl;
    }
};

```

2. 使用虚函数

- 创建一个函数，接受基类指针作为参数，利用多态性调用不同的派生类方法。

```

void renderShape(Shape* shape) {
    shape->draw(); // 调用虚函数
}

int main() {
    Shape* shape1 = new Circle(); // 创建 Circle 对象
    Shape* shape2 = new Square(); // 创建 Square 对象

    renderShape(shape1); // 输出: Drawing Circle
    renderShape(shape2); // 输出: Drawing Square

    delete shape1; // 释放内存
    delete shape2; // 释放内存

    return 0;
}

```

C++中的多态怎么实现的？

C++中的多态主要通过虚函数和继承来实现。多态分为两种：编译时多态和运行时多态。

- 编译时多态：也称为静态多态或早绑定。这种多态是通过函数重载和模板来实现的。
- 运行时多态：也称为动态多态或晚绑定。这种多态是通过虚函数和继承来实现的。当基类的指针或引用指向派生类对象时，调用的虚函数将是派生类的版本，这就实现了运行时多态。

C++多态特性是什么？

多态是面向对象编程（OOP）的重要特性之一，它允许不同的对象对同一消息（函数调用）做出不同的响应。在 C++ 中，多态主要通过虚函数来实现。

多态有静态多态和动态多态两种：

- **静态多态（编译时多态）**：主要通过函数重载来实现。**函数重载**是指在同一个作用域内，可以有多个同名函数，但是它们的参数列表（参数个数、参数类型或者参数顺序）不同。例如：

```
int add(int a, int b) {  
    return a + b;  
}  
  
double add(double a, double b) {  
    return a + b;  
}
```

当调用 `add` 函数时，编译器会根据传入的参数类型和个数在编译时期就确定调用哪个版本的 `add` 函数。

- **动态多态（运行时多态）**：基于虚函数和继承来实现。它允许在运行时根据对象的实际类型来调用相应的函数。当一个类包含虚函数时，编译器会为此类创建一个虚函数表。虚函数表是一个函数指针数组，其中存储了这个类的虚函数的地址。每个包含虚函数的类的对象中都会包含一个虚函数指针，这个指针指向该类的虚函数表。当通过基类指针或引用调用虚函数时，程序会根据虚函数指针找到对应的虚函数表，然后在虚函数表中查找要调用的虚函数的实际地址，从而实现根据对象的实际类型来调用函数。

C++的函数对象是什么？跟普通函数的区别？

函数对象是指一个重载了 `operator()` 的类或结构体实例。函数对象可以像普通函数一样被调用，但它们实际上是对象，具有状态和行为。

普通函数与函数对象的区别：

- 定时方式：普通函数是用 `返回类型 函数名(参数)` 语法定义的，而函数对象是一个类或结构体，并重载了 `operator()`。
- 状态：普通函数无状态，而函数对象可以有内置的状态（成员变量）。
- 调用方式：普通函数被直接调用，函数对象需要先创建实例，然后用实例调用。
- 灵活性：函数对象可以重载多个操作符或添加更多功能，普通函数则只能定义一个函数。

普通函数：

```
int add(int a, int b) {  
    return a + b;  
}
```

函数对象：

```
class Add {  
public:  
    int operator()(int a, int b) {  
        return a + b;  
    }  
};  
  
Add add;  
int result = add(2, 3); // 调用函数对象
```

C++ 类相关

class中缺省的函数是什么？

在C++中，如果一个类没有显式地定义「构造函数、析构函数、拷贝构造函数、赋值运算符重载函数」，那么编译器会自动生成这些函数，这些函数被称为缺省函数。

什么是纯虚函数？有哪些应用场景

纯虚函数是在基类中声明的虚函数，它在基类中没有定义，但要求任何派生类都要定义自己的实现方法。在C++中，纯虚函数的声明形式如下：

```
virtual void function() = 0;
```

其中，`= 0` 就表示这是一个纯虚函数。

含有纯虚函数的类被称为抽象类。抽象类不能被实例化，只能作为接口使用。派生类必须实现所有的纯虚函数，否则该派生类也会变成抽象类。

纯虚函数的应用场景主要包括：

- 设计模式：例如在模板方法模式中，基类定义一个算法的骨架，而将一些步骤延迟到子类中。这些需要在子类中实现的步骤就可以声明为纯虚函数。
- 接口定义：可以创建一个只包含纯虚函数的抽象类作为接口。所有实现该接口的类都必须提供这些函数的实现。

什么是虚函数？什么是纯虚函数？

虚函数：被 `virtual` 关键字修饰的成员函数，就是虚函数。

```
#include <iostream>
using namespace std;

class A
{
public:
    virtual void v_fun() // 虚函数
    {
        cout << "A::v_fun()" << endl;
    }
};

class B : public A
{
public:
    void v_fun()
    {
        cout << "B::v_fun()" << endl;
    }
}
```

```

    }
};
int main()
{
    A *p = new B();
    p->v_fun(); // B::v_fun()
    return 0;
}

```

纯虚函数：

- 纯虚函数在类中声明时，加上 =0；
- 含有纯虚函数的类称为抽象类（只要含有纯虚函数这个类就是抽象类），类中只有接口，没有具体的实现方法；
- 继承纯虚函数的派生类，如果没有完全实现基类纯虚函数，依然是抽象类，不能实例化对象。

说明：

- 抽象类对象不能作为函数的参数，不能创建对象，不能作为函数返回类型；
- 可以声明抽象类指针，可以声明抽象类的引用；
- 子类必须继承父类的纯虚函数，并全部实现后，才能创建子类的对象。

虚函数和纯虚函数的区别？

- 虚函数和纯虚函数可以出现在同一个类中，该类称为抽象基类。（含有纯虚函数的类称为抽象基类）
- 使用方式不同：虚函数可以直接使用，纯虚函数必须在派生类中实现后才能使用；
- 定义形式不同：虚函数在定义时在普通函数的基础上加上 virtual 关键字，纯虚函数定义时除了加上 virtual 关键字还需要加上 =0；
- 虚函数必须实现，否则编译器会报错；
- 对于实现纯虚函数的派生类，该纯虚函数在派生类中被称为虚函数，虚函数和纯虚函数都可以在派生类中重写；
- 析构函数最好定义为虚函数，特别是对于含有继承关系的类；析构函数可以定义为纯虚函数，此时，其所在的类为抽象基类，不能创建实例化对象。

虚函数的实现机制是什么？

实现机制：虚函数通过虚函数表来实现。虚函数的地址保存在虚函数表中，在类的对象所在的内存空间中，保存了指向虚函数表的指针（称为“虚表指针”），通过虚表指针可以找到类对应的虚函数表。虚函数表解决了基类和派生类的继承问题和类中成员函数的覆盖问题，当用基类的指针来操作一个派生类的时候，这张虚函数表就指明了实际应该调用的函数

虚函数表相关知识点：

- 虚函数表存放的内容：类的虚函数的地址。
- 虚函数表建立的时间：编译阶段，即程序的编译过程中会将虚函数的地址放在虚函数表中。
- 虚表指针保存的位置：虚表指针存放在对象的内存空间中最前面的位置，这是为了保证正确取到虚函数的偏移量。

注：虚函数表和类绑定，虚表指针和对象绑定。即类的不同的对象的虚函数表是一样的，但是每个对象都有自己的虚表指针，来指向类的虚函数表。

例子，无虚函数覆盖的情况：

```
#include <iostream>
using namespace std;

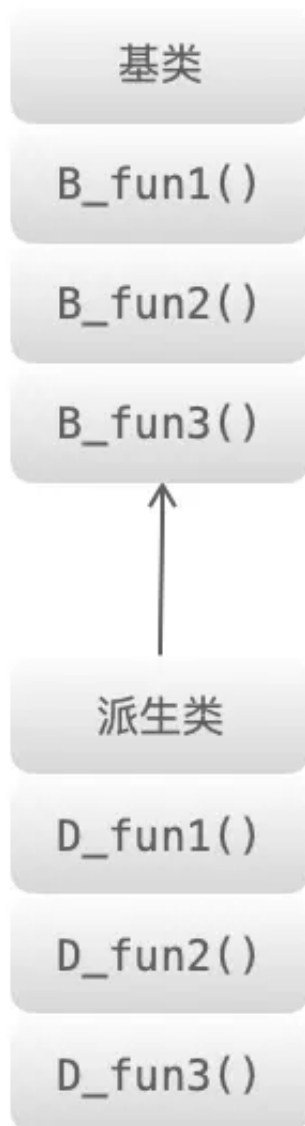
class Base
{
public:
    virtual void B_fun1() { cout << "Base::B_fun1()" << endl; }
    virtual void B_fun2() { cout << "Base::B_fun2()" << endl; }
    virtual void B_fun3() { cout << "Base::B_fun3()" << endl; }
};

class Derive : public Base
{
public:
    virtual void D_fun1() { cout << "Derive::D_fun1()" << endl; }
    virtual void D_fun2() { cout << "Derive::D_fun2()" << endl; }
    virtual void D_fun3() { cout << "Derive::D_fun3()" << endl; }
};

int main()
{
    Base *p = new Derive();
    p->B_fun1(); // Base::B_fun1()
    return 0;
}
```

}

基类和派生类的继承关系：



基类的虚函数表：

基类的对象

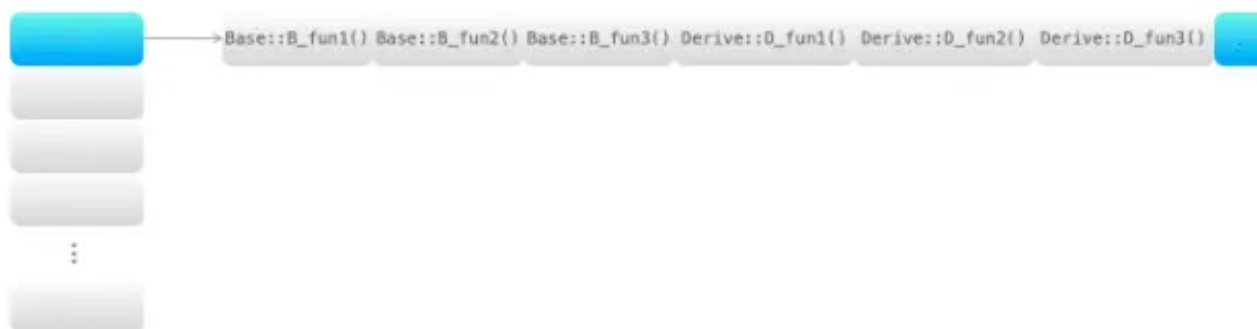
虚函数表



派生类的虚函数表：

派生类的对象

虚函数表



主函数中基类的指针 p 指向了派生类的对象，当调用函数 B_fun1() 时，通过派生类的虚函数表找到该函数的地址，从而完成调用。

单继承和多继承的虚函数表结构是怎样的？

编译器处理虚函数表：

- 编译器将虚函数表的指针放在类的实例对象的内存空间中，该对象调用该类的虚函数时，通过指针找到虚函数表，根据虚函数表中存放的虚函数的地址找到对应的虚函数。
- 如果派生类没有重新定义基类的虚函数 A，则派生类的虚函数表中保存的是基类的虚函数 A 的地址，也就是说基类和派生类的虚函数 A 的地址是一样的。
- 如果派生类重写了基类的某个虚函数 B，则派生的虚函数表中保存的是重写后的虚函数 B 的地址，也就是说虚函数 B 有两个版本，分别存放在基类和派生类的虚函数表中。
- 如果派生类重新定义了新的虚函数 C，派生类的虚函数表保存新的虚函数 C 的地址。

单继承无虚函数覆盖的情况：

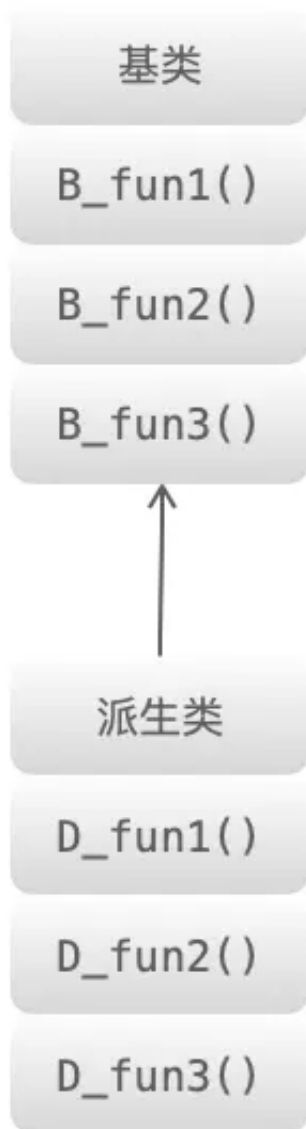
```
#include <iostream>
using namespace std;

class Base
{
public:
    virtual void B_fun1() { cout << "Base::B_fun1()" << endl; }
    virtual void B_fun2() { cout << "Base::B_fun2()" << endl; }
    virtual void B_fun3() { cout << "Base::B_fun3()" << endl; }
};

class Derive : public Base
{
public:
    virtual void D_fun1() { cout << "Derive::D_fun1()" << endl; }
    virtual void D_fun2() { cout << "Derive::D_fun2()" << endl; }
    virtual void D_fun3() { cout << "Derive::D_fun3()" << endl; }
};

int main()
{
    Base *p = new Derive();
    p->B_fun1(); // Base::B_fun1()
    return 0;
}
```

基类和派生类的继承关系：



基类的虚函数表：

基类的对象

虚函数表



派生类的虚函数表：

派生类的对象

虚函数表



单继承有虚函数覆盖的情况：

```
#include <iostream>
using namespace std;

class Base
{
public:
    virtual void fun1() { cout << "Base::fun1()" << endl; }
    virtual void B_fun2() { cout << "Base::B_fun2()" << endl; }
    virtual void B_fun3() { cout << "Base::B_fun3()" << endl; }
};

class Derive : public Base
{
public:
    virtual void fun1() { cout << "Derive::fun1()" << endl; }
    virtual void D_fun2() { cout << "Derive::D_fun2()" << endl; }
    virtual void D_fun3() { cout << "Derive::D_fun3()" << endl; }
};

int main()
{
    Base *p = new Derive();
    p->fun1(); // Derive::fun1()
    return 0;
}
```

派生类的虚函数表：

派生类的对象

虚函数表



多继承无虚函数覆盖的情况：

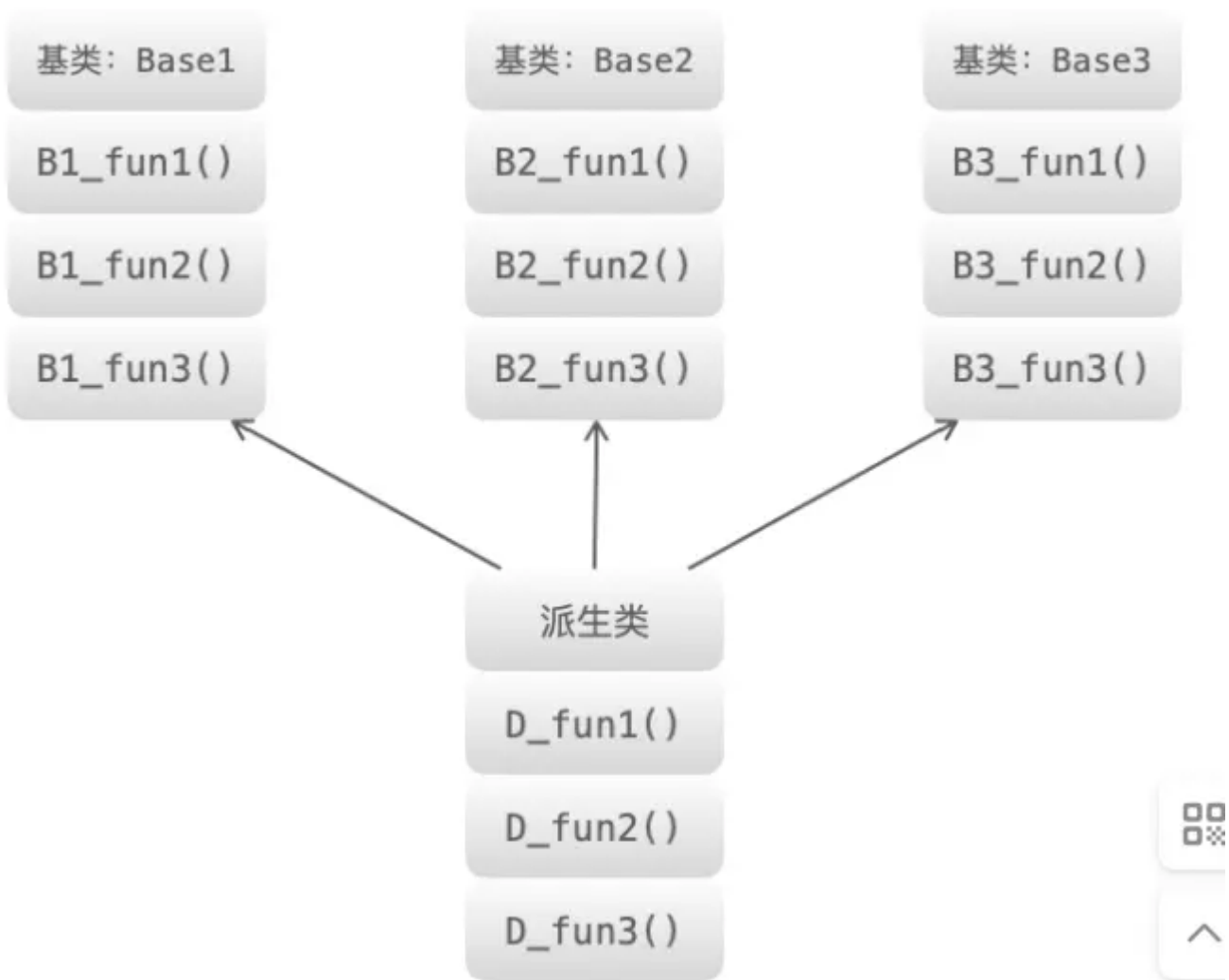
```
#include <iostream>
using namespace std;

class Base
{
public:
    virtual void fun1() { cout << "Base::fun1()" << endl; }
    virtual void B_fun2() { cout << "Base::B_fun2()" << endl; }
    virtual void B_fun3() { cout << "Base::B_fun3()" << endl; }
};

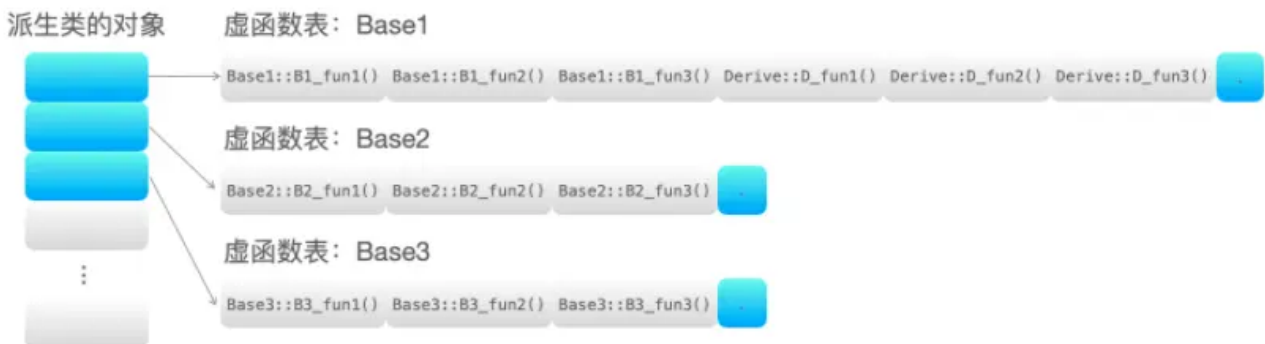
class Derive : public Base
{
public:
    virtual void fun1() { cout << "Derive::fun1()" << endl; }
    virtual void D_fun2() { cout << "Derive::D_fun2()" << endl; }
    virtual void D_fun3() { cout << "Derive::D_fun3()" << endl; }
};

int main()
{
    Base *p = new Derive();
    p->fun1(); // Derive::fun1()
    return 0;
}
```

基类和派生类的关系：



派生类的虚函数表：（基类的顺序和声明的顺序一致）



多继承有虚函数覆盖的情况：

```

#include <iostream>
using namespace std;

class Base1
{
public:
    virtual void B1_fun1() { cout << "Base1::B1_fun1()" << endl; }
  
```

```

        virtual void B1_fun2() { cout << "Base1::B1_fun2()" << endl; }
        virtual void B1_fun3() { cout << "Base1::B1_fun3()" << endl; }
};

class Base2
{
public:
    virtual void B2_fun1() { cout << "Base2::B2_fun1()" << endl; }
    virtual void B2_fun2() { cout << "Base2::B2_fun2()" << endl; }
    virtual void B2_fun3() { cout << "Base2::B2_fun3()" << endl; }
};

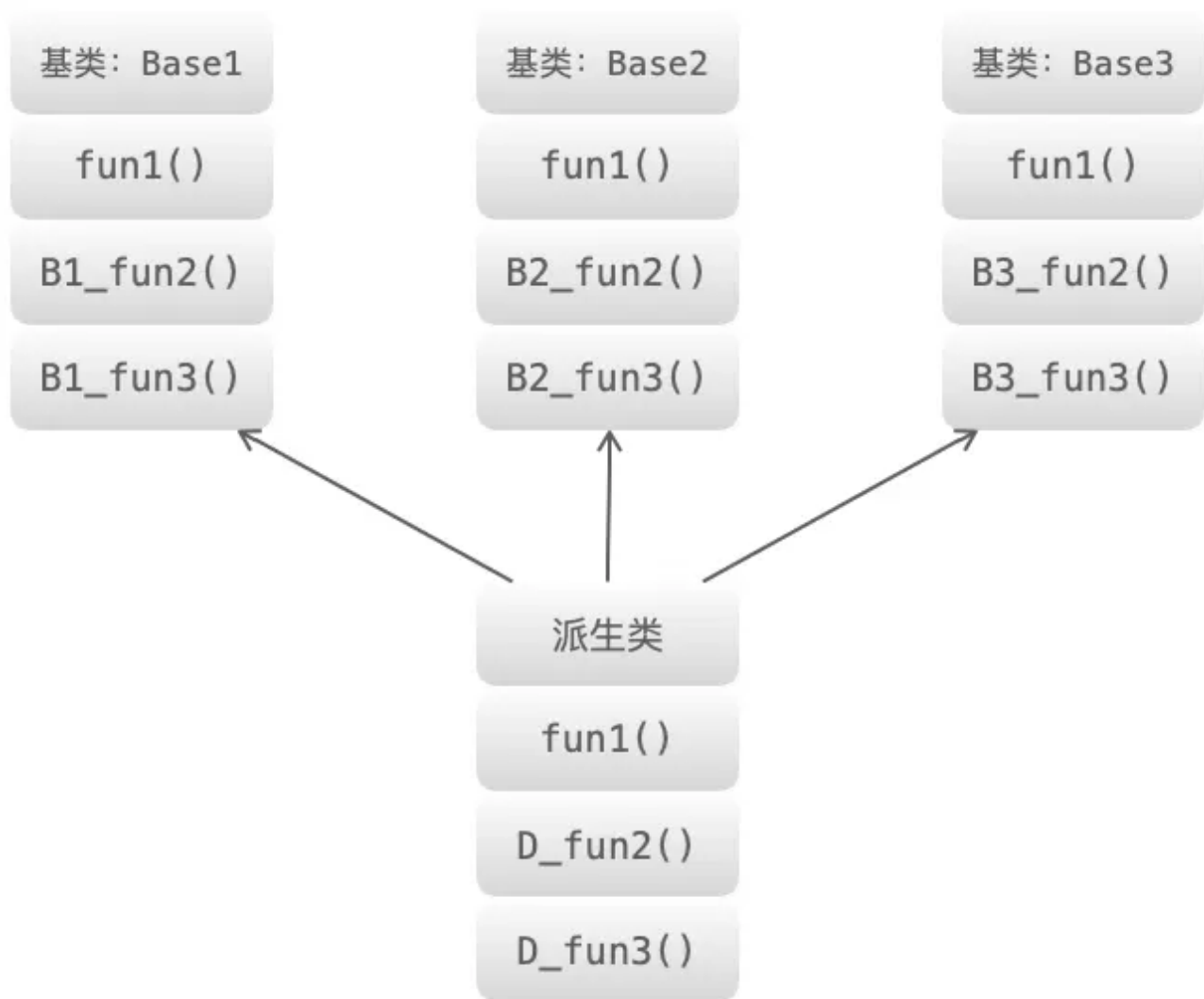
class Base3
{
public:
    virtual void B3_fun1() { cout << "Base3::B3_fun1()" << endl; }
    virtual void B3_fun2() { cout << "Base3::B3_fun2()" << endl; }
    virtual void B3_fun3() { cout << "Base3::B3_fun3()" << endl; }
};

class Derive : public Base1, public Base2, public Base3
{
public:
    virtual void D_fun1() { cout << "Derive::D_fun1()" << endl; }
    virtual void D_fun2() { cout << "Derive::D_fun2()" << endl; }
    virtual void D_fun3() { cout << "Derive::D_fun3()" << endl; }
};

int main(){
    Base1 *p = new Derive();
    p->B1_fun1(); // Base1::B1_fun1()
    return 0;
}

```

基类和派生类的关系：



派生类的虚函数表:



C++空类的大小是多少?

C++ 空类的大小是 **1 字节**。

原因很简单: C++ 规定, 任何对象都必须有一个唯一的内存地址。如果空类大小为 0, 那么当创建这个类的多个对象时, 它们会共享同一个地址, 这违反了“每个对象地址唯一”的

规则。所以编译器会给空类隐式分配 1 字节的空间，目的就是为了让这个类的每个实例都能拥有独一无二的内存地址。

例如：

```
class A {};  
int main(){  
    cout<<sizeof(A)<<endl;// 输出 1;  
    A a;  
    cout<<sizeof(a)<<endl;// 输出 1;  
    return 0;  
}
```

1. C++空类的大小不为0，不同编译器设置不一样，**vs和lg++都是设置为1**；
2. C++标准指出，不允许一个对象（当然包括类对象）的大小为0，不同的对象不能具有相同的地址；
3. 带有虚函数的C++类大小不为1**，因为每一个对象会有一个vp_{tr}指向虚函数表，**具体大小根据指针大小确定；
4. C++中要求对于类的每个实例都必须有独一无二的地址,那么编译器自动为空类分配一个字节大小，这样便保证了每个实例均有独一无二的内存地址。

在C++中空类会占一个字节，这是为了让对象的实例能够相互区别。具体来说，空类同样可以被实例化，并且每个实例在内存中都有独一无二的地址，因此，编译器会给空类隐含加上一个字节，这样空类实例化之后就会拥有独一无二的内存地址。当该空白类作为基类时，该类的大小就优化为0了，子类的大小就是子类本身的大小。这就是所谓的空白基类最优化。

空类的实例大小就是类的大小，所以sizeof(a)=1字节，如果a是指针，则sizeof(a)就是指针的大小，即4字节。

只含虚函数的类的大小是多大？

因为有虚函数的类对象中都有一个**虚函数表指针 __vp_{tr}**，其大小是4字节（32位机器上）

```
class A { virtual Fun(){} };  
int main(){  
    cout<<sizeof(A)<<endl;// 输出 4(32位机器)/8(64位机器);  
    A a;  
    cout<<sizeof(a)<<endl;// 输出 4(32位机器)/8(64位机器);  
}
```

```
    return 0;
}
```

一个只包含int 变量的空class和只包含int变量的空struct的内存各占多大？

只含有一个int成员变量的类的大小是4

```
class A { int a; };
int main(){
    cout<<sizeof(A)<<endl;// 输出 4;
    A a;
    cout<<sizeof(a)<<endl;// 输出 4;
    return 0;
}
```

只是一个int变量的大小——4字节

只含有一个静态成员变量的类的大小是1

```
class A { static int a; };
int main(){
    cout<<sizeof(A)<<endl;// 输出 1;
    A a;
    cout<<sizeof(a)<<endl;// 输出 1;
    return 0;
}
```

java

静态成员存放在静态存储区，不占用类的大小，普通函数也不占用类大小

```
class A { static int a; int b; };
int main(){
    cout<<sizeof(A)<<endl;// 输出 4;
    A a;
    cout<<sizeof(a)<<endl;// 输出 4;
    return 0;
}
```

java

静态成员a不占用类的大小，所以类的大小就是b变量的大小 即4个字节

C++ 语言特性

左值和右值的区别？左值引用和右值引用的区别，如何将左值转换成右值？

左值：指表达式结束后依然存在的持久对象。

右值：表达式结束就不再存在的临时对象。

左值和右值的区别：左值持久，右值短暂

右值引用和左值引用的区别：

- 左值引用不能绑定到要转换的表达式、字面常量或返回右值的表达式。右值引用恰好相反，可以绑定到这类表达式，但不能绑定到一个左值上。
- 右值引用必须绑定到右值的引用，通过 && 获得。右值引用只能绑定到一个将要销毁的对象上，因此可以自由地移动其资源。

std::move 可以将一个左值强制转化为右值，继而可以通过右值引用使用该值，以用于移动语义。

```
#include <iostream>
using namespace std;

void fun1(int& tmp)
{
    cout << "fun1(int& tmp):" << tmp << endl;
}

void fun2(int&& tmp)
{
    cout << "fun2(int&& tmp)" << tmp << endl;
}

int main()
{
    int var = 11;
    fun1(12); // error: cannot bind non-const lvalue reference of type 'in
```

```
    fun1(var);  
    fun2(1);  
}
```

介绍移动语义

移动语义通过右值引用（Rvalue References）和移动构造函数（Move Constructors）来优化对象在内存中的传递和处理，避免不必要的数据复制。

传统的拷贝操作对于大型对象或资源密集型对象来说可能会有很高的开销，因为它们需要将对象的所有数据复制到新的对象中。移动语义的引入解决了这一问题，实现了对象资源的“移动”而非“复制”。

- **右值引用（Rvalue References）**：通过使用双引号&&声明右值引用，可以绑定到临时对象或右值，即那些将会被销毁的临时对象。右值引用允许我们对其资源进行移动操作而不是复制操作。

```
T&& t = T(); // t为右值引用
```

- **移动构造函数（Move Constructor）**：移动构造函数是专门用于将对象资源从一个右值引用对象“移动”到另一个对象的构造函数。它通过将资源所有权从一个对象转移到另一个对象来避免不必要的数据复制，从而提高程序性能。

```
// 移动构造函数示例  
T(T&& other) {  
    // 将other对象的资源移动到当前对象  
}
```

- **标记对象为右值（std::move）**：使用std::move函数可以将一个对象标记为右值，以便可以调用移动构造函数而不是拷贝构造函数。

```
T obj1;  
T obj2 = std::move(obj1); // 将obj1标记为右值，调用移动构造函数
```

移动语义的应用可以在涉及对象所有权转移的情况下提高性能，特别是在动态内存管理和容器元素移动时。通过避免不必要的复制操作，移动语义使得代码更加高效和可维护，同时减

少了资源的浪费。

右值引用有什么作用？

- 右值引用是C++11引入的特性，它是指对右值进行引用的一种方式。右值引用的作用主要有两个：
- 可以通过右值引用来实现移动语义。移动语义可以在不进行深拷贝的情况下，将对象的资源所有权从一个对象转移到另一个对象，从而提高代码的效率。
- 右值引用还可以用于完美转发。在函数模板中，通过使用右值引用类型的形参来接收参数，可以实现完美转发，即保持原参数的值类别（左值还是右值），将参数传递给另一个函数。

std::move() 函数的实现原理是什么？

std::move() 函数原型：

```
template <typename T>
typename remove_reference<T>::type&& move(T&& t) {
    return static_cast<typename remove_reference<T>::type &&>(t);
}
```

说明：引用折叠原理

- 右值传递给上述函数的形参 T&& 依然是右值，即 T&& && 相当于 T&&。
- 左值传递给上述函数的形参 T&& 依然是左值，即 T&& & 相当于 T&。

小结：通过引用折叠原理可以知道，move() 函数的形参既可以是左值也可以是右值。

remove_reference 具体实现：

```
//原始的，最通用的版本
template <typename T>
struct remove_reference {
    typedef T type; //定义 T 的类型别名为 type
};

//部分版本特例化，将用于左值引用和右值引用
template <class T>
```

```

struct remove_reference<T&> { //左值引用
    typedef T type;
};

template <class T>
struct remove_reference<T&&> { //右值引用
    typedef T type;
};

//举例如下,下列定义的a、b、c三个变量都是int类型
int i;
remove_reference<decltype(42)>::type a; //使用原版本,
remove_reference<decltype(i)>::type b; //左值引用特例版本
remove_reference<decltype(std::move(i))>::type c; //右值引用特例版本

```

举例：

```
int var = 10;
```

转化过程：

1. `std::move(var)` => `std::move(int&& &)` => 折叠后 `std::move(int&)`
2. 此时：T 的类型为 `int&`，`typename remove_reference<T>::type` 为 `int`，这里使用
3. 通过 `static_cast` 将 `int&` 强制转换为 `int&&`

整个`std::move`被实例化如下：

```

string&& move(int& t) {
    return static_cast<int&&>(t);
}

```

`std::move()` 实现原理：

- 利用引用折叠原理将右值经过 `T&&` 传递类型保持不变还是右值，而左值经过 `T&&` 变为普通的左值引用，以保证模板可以传递任意实参，且保持类型不变；
- 然后通过 `remove_refrence` 移除引用，得到具体的类型 `T`；
- 最后通过 `static_cast<>` 进行强制类型转换，返回 `T&&` 右值引用。

如何判断结构体是否相等？能否用 `memcmp` 函数判断结构体相等？

需要重载操作符 `==` 判断两个结构体是否相等，不能用函数 `memcmp` 来判断两个结构体是

否相等，因为 memcmp 函数是逐个字节进行比较的，而结构体存在内存空间中保存时存在字节对齐，字节对齐时补的字节内容是随机的，会产生垃圾值，所以无法比较。

利用运算符重载来实现结构体对象的比较：

```
#include <iostream>

using namespace std;

struct A
{
    char c;
    int val;
    A(char c_tmp, int tmp) : c(c_tmp), val(tmp) {}

    friend bool operator==(const A &tmp1, const A &tmp2); // 友元运算符重
};

bool operator==(const A &tmp1, const A &tmp2)
{
    return (tmp1.c == tmp2.c && tmp1.val == tmp2.val);
}

int main()
{
    A ex1('a', 90), ex2('b', 80);
    if (ex1 == ex2)
        cout << "ex1 == ex2" << endl;
    else
        cout << "ex1 != ex2" << endl; // 输出
    return 0;
}
```

什么是模板？如何实现？

模板：创建类或者函数的蓝图或者公式，分为函数模板和类模板。实现方式：模板定义以关键字 template 开始，后跟一个模板参数列表。

- 模板参数列表不能为空；
- 模板类型参数前必须使用关键字 class 或者 typename，在模板参数列表中这两个关键字含义相同，可互换使用。

```
template <typename T, typename U, ...>
```

函数模板：通过定义一个函数模板，可以避免为每一种类型定义一个新函数。

- 对于函数模板而言，模板类型参数可以用来指定返回类型或函数的参数类型，以及在函数体内用于变量声明或类型转换。
- 函数模板实例化：当调用一个模板时，编译器用函数实参来推断模板实参，从而使用实参的类型来确定绑定到模板参数的类型。

```
#include <iostream>
using namespace std;

template <typename T>
T add_fun(const T & tmp1, const T & tmp2) {
    return tmp1 + tmp2;
}

int main() {
    int var1, var2;
    cin >> var1 >> var2;
    cout << add_fun(var1, var2);

    double var3, var4;
    cin >> var3 >> var4;
    cout << add_fun(var3, var4);
    return 0;
}
```

类模板：类似函数模板，类模板以关键字 `template` 开始，后跟模板参数列表。但是，编译器不能为类模板推断模板参数类型，需要在使用该类模板时，在模板名后面的尖括号中指明类型。

```
#include <iostream>
using namespace std;

template <typename T>
class Complex {
public:
    //构造函数
```

```

Complex(T a, T b) {
    this->a = a;
    this->b = b;
}

//运算符重载
Complex<T> operator+(Complex &c) {
    Complex<T> tmp(this->a + c.a, this->b + c.b);
    cout << tmp.a << " " << tmp.b << endl;
    return tmp;
}
private:
    T a;
    T b;
};

int main() {
    Complex<int> a(10, 20);
    Complex<int> b(20, 30);
    Complex<int> c = a + b;
    return 0;
}

```

函数模板和类模板的区别？

- 实例化方式不同：函数模板实例化由编译程序在处理函数调用时自动完成，类模板实例化需要在程序中显式指定。
- 实例化的结果不同：函数模板实例化后是一个函数，类模板实例化后是一个类。
- 默认参数：类模板在模板参数列表中可以有默认参数。
- 特化：函数模板只能全特化；而类模板可以全特化，也可以偏特化。
- 调用方式不同：函数模板可以隐式调用，也可以显式调用；类模板只能显式调用。

函数模板调用方式举例：

```

#include <iostream>
using namespace std;

template <typename T>
T add_fun(const T & tmp1, const T & tmp2) {
    return tmp1 + tmp2;
}

```

```

}

int main() {
    int var1, var2;
    cin >> var1 >> var2;
    cout << add_fun<int>(var1, var2); // 显式调用

    double var3, var4;
    cin >> var3 >> var4;
    cout << add_fun(var3, var4); // 隐式调用
    return 0;
}

```

什么是可变参数模板？ 可变参数模板：接受可变数目参数的模板函数或模板类。将可变数目的参数被称为参数包，包括模板参数包和函数参数包。

- 模板参数包：表示零个或多个模板参数；
- 函数参数包：表示零个或多个函数参数。

用省略号来指出一个模板参数或函数参数表示一个包，在模板参数列表中，class... 或 typename... 指出接下来的参数表示零个或多个类型的列表；一个类型名后面跟一个省略号表示零个或多个给定类型的非类型参数的列表。当需要知道包中有多少元素时，可以使用 sizeof... 运算符。

```

template <typename T, typename... Args> // Args 是模板参数包
void foo(const T &t, const Args&... rest); // 可变参数模板, rest 是函数参数包

#include <iostream>
using namespace std;

template <typename T>
void print_fun(const T &t) {
    cout << t << endl; // 最后一个元素
}

template <typename T, typename... Args>
void print_fun(const T &t, const Args &...args) {
    cout << t << " ";
    print_fun(args...);
}

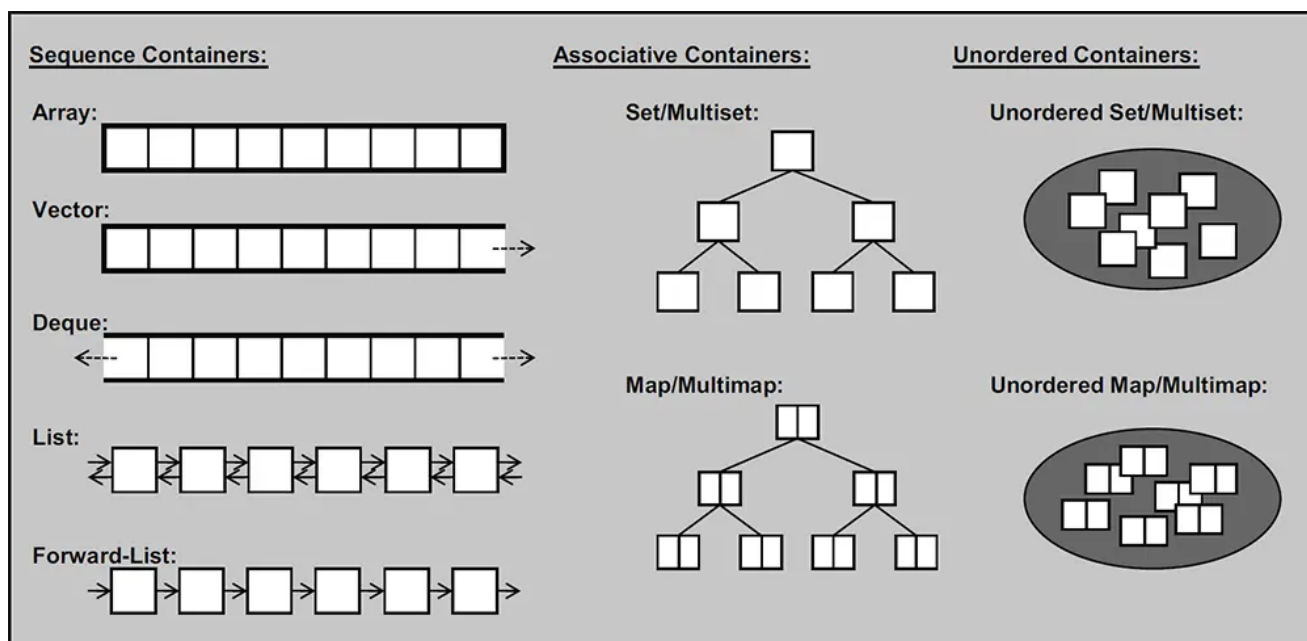
```

```
int main() {
    print_fun("Hello", "world", "!");
    return 0;
}
/*运行结果：
Hello world !
*/
```

说明：可变参数函数通常是递归的，第一个版本的 `print_fun` 负责终止递归并打印初始调用中的最后一个实参。第二个版本的 `print_fun` 是可变参数版本，打印绑定到 `t` 的实参，并用来调用自身来打印函数参数包中的剩余值。

C++ STL

C++ STL（标准模板库）提供了多种容器，用于存储和管理数据，这些容器可分为**序列式容器**、**关联式容器**和**容器适配器**三大类，各自适用于不同场景：



序列式容器：按元素插入顺序存储，元素位置与插入顺序相关，不自动排序。

- **vector**：动态数组，内存连续，支持随机访问（`[]` 或 `at()`），优点尾插 / 尾删效率高（ $O(1)$ ），适合频繁访问元素的场景。缺点是中间插入 / 删除效率低（ $O(n)$ ），扩容时可能重新分配内存。
- **deque**：双端队列，内存分段连续，支持首尾高效操作。优点头插 / 头删、尾插 / 尾删效率均为 $O(1)$ ，可随机访问，适合场景需要在两端频繁操作的场景（如实现队列、栈）。

- `list` : 双向链表, 元素通过指针连接, 内存不连续。优点任意位置插入 / 删除效率高 ($O(1)$, 只需修改指针)。缺点不支持随机访问 (访问元素需遍历, $O(n)$), 内存开销较大。
- `array` : 固定大小数组 (C++11 新增), 编译时确定大小, 内存连续, 比原生数组更安全 (支持边界检查), 但大小不可变。

关联式容器: 元素按键 (key) 排序存储, 支持快速查找 (通常 $O(\log n)$), 分为有序和无序两类。

有序关联容器 (基于红黑树实现)

- `set` : 存储唯一键值, 元素自动按键升序排序, 键即值 (`key=value`)。适合场景需要去重且有序的数据集合 (如存储不重复的 ID 并排序)。
- `multiset` : 与 `set` 类似, 但允许键值重复, 其他特性相同。
- `map` : 存储键值对 (`key-value`), 键唯一且自动排序, 通过键快速查找值。适合场景键值映射场景 (如字典、配置表)。
- `multimap` : 与 `map` 类似, 但允许键重复 (一个键可对应多个值)。

无序关联容器 (C++11 新增, 基于哈希表实现)

- `unordered_set` / `unordered_multiset` : 功能同 `set` / `multiset`, 但元素无序, 通过哈希表实现, 查找、插入、删除平均效率更高 ($O(1)$)。适合场景对顺序无要求, 但需要快速增删查的场景。
- `unordered_map` / `unordered_multimap` : 功能同 `map` / `multimap`, 无序, 基于哈希表, 平均操作效率 $O(1)$ 。

容器适配器: 基于其他容器实现, 封装特定接口, 提供受限功能。

- `stack` : 栈, 遵循“后进先出 (LIFO)”, 仅支持在顶部插入 / 删除 / 访问元素, 默认基于 `deque` 实现, 也可指定 `vector` 或 `list` 作为底层容器。
- `queue` : 队列, 遵循“先进先出 (FIFO)”, 仅支持在尾部插入、头部删除。默认基于 `deque` 实现, 也可指定 `list` 作为底层容器。
- `priority_queue` : **优先队列, 元素按优先级自动排序 (默认最大元素在顶部), 插入 / 删除效率 $O(\log n)$ 。默认基于 `vector` 实现, 底层用堆结构维护优先级。

vector中push_back和emplace_back的区别?

- `push_back()` 向容器尾部添加元素时, 首先会创建这个元素, 然后再将这个元素拷贝或

者移动到容器中（如果是拷贝的话，事后会自行销毁先前创建的这个元素）；

- 而`emplace_back()` 在实现时，则是直接在容器尾部创建这个元素，省去了拷贝或移动元素的过程。

C++ 的map是线程安全的么？

不是线程安全的。如果多个线程同时对 `std::map` 进行操作（尤其是写操作，如插入、删除、修改元素），可能会导致未定义行为（例如数据损坏、迭代器失效、程序崩溃等）。

具体来说，线程不安全主要体现在：

1. **并发写操作**：多个线程同时执行插入（`insert`）、删除（`erase`）或修改（如 `operator[]` 赋值）时，会破坏 `std::map` 内部的红黑树结构（有序关联容器的底层实现），导致数据错乱。
2. **读写并发**：即使一个线程读、另一个线程写，也可能出现问题。例如，读线程正在遍历 `map` 时，写线程修改了结构，可能导致读线程的迭代器失效，触发不可预知的错误。
3. **无内置同步机制**：`std::map` 没有提供任何锁（如互斥量）或原子操作来保证多线程安全，所有同步逻辑需要开发者手动实现。

如果要多线程安全使用`map`，需通过**外部同步机制**保证线程安全，常见方式：

- **加锁保护**：使用 `std::mutex` 或 `std::lock_guard` 等同步工具，确保同一时间只有一个线程能访问或修改 `map`。例如：

```
#include <map>
#include <mutex>

std::map<int, int> my_map;
std::mutex mtx; // 互斥锁

// 线程安全的插入操作
void safe_insert(int key, int value) {
    std::lock_guard<std::mutex> lock(mtx); // 自动加锁/解锁
    my_map[key] = value;
}

// 线程安全的查找操作
int safe_find(int key) {
    std::lock_guard<std::mutex> lock(mtx);
    auto it = my_map.find(key);
```

```

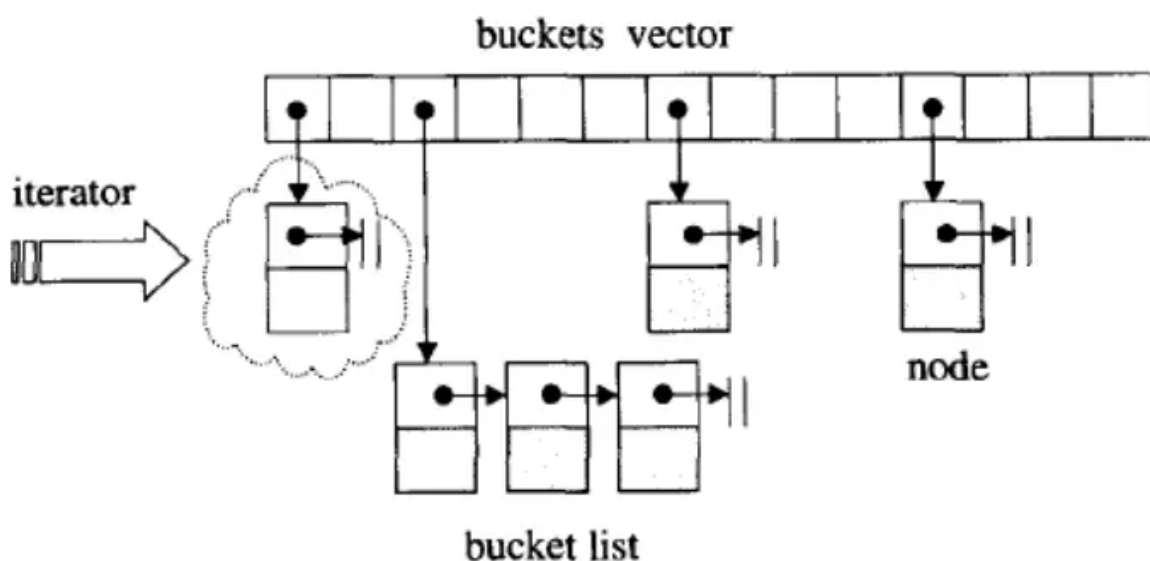
    return (it != my_map.end()) ? it->second : -1;
}

```

- **使用线程安全的替代容器**：某些库（如 C++17 后的 `concurrent_unordered_map` 非标准扩展，或第三方库如 Intel TBB）提供了线程安全的哈希表 / 映射容器，可减少手动加锁的开销。

unordered_map的底层结构是什么？

底层结构基于哈希表实现。



以开链（separate chaining）法完成的 hash table。

`std::unordered_map` 内部维护了一个哈希桶数组，数组中的每个元素称为一个哈希桶。每个哈希桶可以存储一个或多个键值对。当多个键通过哈希函数计算得到相同的索引时，就会发生哈希冲突。

`std::unordered_map` 通常使用链地址法来解决哈希冲突。在链地址法中，每个哈希桶是一个链表，当发生哈希冲突时，新的键值对会被插入到对应的链表或容器中。

C++容器可以一边遍历一边插入吗？

当处理 `vector`, `string`, `deque` 时，当在一个循环中可能增加或移除元素时，要考虑到迭代器可能会失效的问题。

`vector` 插入的时候：

- 当插入(push_back)一个元素后，end操作返回的迭代器肯定失效；
- 当插入(push_back)一个元素后，如果vector的capacity发生了改变，则需要重新加载整个容器，此时first和end操作返回的迭代器都会失效；

list插入的时候：

- 插入操作(insert)和接合操作(splice)不会造成原有的list迭代器失效，

deque插入的时候：

- 在deque容器首部或者尾部插入元素，不会使得任何迭代器失效；
- 在deque容器的任何其他位置进行插入或删除操作都将使指向该容器元素的所有迭代器失效；

set和map插入的时候：

- 与list相同，当对其进行insert或者erase操作时，操作之前的所有迭代器，在操作完成之后都依然有效，但被删除元素的迭代器失效。

使用迭代器怎么删除一个元素？

顺序容器（序列式容器，比如vector、deque）删除元素的方式：erase迭代器不仅使所指向被删除的迭代器失效，而且使被删元素之后的所有迭代器失效(list除外)，所以不能使用erase(it++)的方式，但是erase的返回值是下一个有效迭代器：It = c.erase(it);

java

```
std::vector<int> arrayInt;
...
std::vector<int>::iterator it = arrayInt.begin();
while (it != arrayInt.end())
{
    if (...)
    {
        // 需要注意的是，因为顺序式容器会使本身和后面的元素迭代器都失效，所以不能简单
        // 顺序式容器的erase()会返回紧随被删除元素的下一个元素的有效迭代器（节点式容
        it = arrayInt.erase(it);
    }
    else
    {
        it++;
    }
}
```

关联容器(关联式容器，比如map、set、multimap、multiset等) 删除元素的方式：erase
迭代器只是被删除元素的迭代器失效，但是返回值是void，所以要采用erase(it++)的方式
删除迭代器；c.erase(it++)

```
std::map<int, struct> mapInfo;
...
std::map<int, struct>::iterator it = mapInfo.begin();
while (it != mapInfo.end())
{
    if (...)
    {
        // 删除节点的前，对迭代器进行后移的操作，因为其他元素不会失效
        mapInfo.erase(it++);
    }
    else
    {
        it++;
    }
}
```

stl迭代器的失效情况你知道哪些？

序列式容器（vector、deque），迭代器失效的情况

当在vector中插入元素时，如果插入操作导致容器的内存重新分配（即插入后容器的容量不足，需要重新分配更大的内存空间），那么所有指向vector的迭代器、指针和引用都会失效。因为重新分配内存后，元素会被移动到新的内存位置，例子代码：

```
#include <iostream>
#include <vector>

int main() {
    std::vector<int> vec = {1, 2, 3};
    auto it = vec.begin();
    vec.push_back(4); // 可能导致内存重新分配
    // 此时 it 可能已经失效
    // std::cout << *it << std::endl; // 未定义行为
    return 0;
}
```

```
}
```

当在 `vector` 中删除元素时，指向被删除元素的迭代器、指针和引用会失效，并且指向删除位置之后的元素的迭代器、指针和引用也会失效。代码如下：

```
#include <iostream>
#include <vector>

int main() {
    std::vector<int> vec = {1, 2, 3};
    auto it = vec.begin() + 1;
    vec.erase(vec.begin()); // 删除第一个元素
    // 此时 it 失效
    // std::cout << *it << std::endl; // 未定义行为
    return 0;
}
```

在 `deque` 的中间插入元素时，所有迭代器、指针和引用都会失效，还有在 `deque` 的头部或尾部插入元素时，指向元素的迭代器、指针和引用不会失效，但如果插入操作导致内存重新分配，那么迭代器可能会失效。

```
#include <iostream>
#include <deque>

int main() {
    std::deque<int> deq = {1, 2, 3};
    auto it = deq.begin() + 1;
    deq.insert(deq.begin() + 1, 4); // 在中间插入元素
    // 此时 it 失效
    // std::cout << *it << std::endl; // 未定义行为
    return 0;
}
```

删除 `deque` 中间的元素时，所有迭代器、指针和引用都会失效，还有删除 `deque` 头部或尾部的元素时，指向被删除元素的迭代器、指针和引用会失效。

```
#include <iostream>
#include <deque>
```

```
int main() {
    std::deque<int> deq = {1, 2, 3};
    auto it = deq.begin() + 1;
    deq.erase(deq.begin() + 1); // 删除中间元素
    // 此时 it 失效
    // std::cout << *it << std::endl; // 未定义行为
    return 0;
}
```

关联式容器（set、map），迭代器失效的情况

set 在插入元素不会使任何迭代器、指针和引用失效，因为关联式容器使用红黑树等平衡二叉搜索树实现，插入操作只是在树中添加新节点，不会影响其他节点的内存位置。

不过指向被删除元素的迭代器、指针和引用会失效，其他迭代器、指针和引用不会失效。

```
#include <iostream>
#include <set>

int main() {
    std::set<int> s = {1, 2, 3};
    auto it = s.begin();
    auto it_to_delete = s.find(2);
    s.erase(it_to_delete);
    // 此时 it_to_delete 失效
    // std::cout << *it_to_delete << std::endl; // 未定义行为
    std::cout << *it << std::endl; // 正常输出
    return 0;
}
```

map 在插入元素不会使任何迭代器、指针和引用失效，原因与 set。但是指向被删除元素的迭代器、指针和引用会失效，其他迭代器、指针和引用不会失效。

```
#include <iostream>
#include <map>

int main() {
    std::map<int, int> m = {{1, 10}, {2, 20}, {3, 30}};
    auto it = m.begin();
    auto it_to_delete = m.find(2);
    m.erase(it_to_delete);
}
```

```

// 此时 it_to_delete 失效
// std::cout << it_to_delete->second << std::endl; // 未定义行为
std::cout << it->second << std::endl; // 正常输出
return 0;
}

```

链表式容器（list），迭代器失效

在 list 插入元素不会使任何迭代器、指针和引用失效，因为链表的插入操作只是修改节点的指针，不会影响其他节点的内存位置。但是，指向被删除元素的迭代器、指针和引用会失效，其他迭代器、指针和引用不会失效。

```

#include <iostream>
#include <list>

int main() {
    std::list<int> lst = {1, 2, 3};
    auto it = lst.begin();
    auto it_to_delete = ++lst.begin();
    lst.erase(it_to_delete);
    // 此时 it_to_delete 失效
    // std::cout << *it_to_delete << std::endl; // 未定义行为
    std::cout << *it << std::endl; // 正常输出
    return 0;
}

```

STL 容器中优先级队列 priority_queue 的底层原理

priority_queue 优先级队列之所以总能保证优先级最高的元素位于队头，最重要的原因是其底层采用堆数据结构存储结构。

有读者可能会问，priority_queue 底层不是采用 vector 或 deque 容器存储数据吗，这里又说使用堆结构存储数据，它们之间不冲突吗？显然，它们之间是不冲突的。

priority_queue 底层采用 vector 或 deque 容器存储数据和采用堆数据结构是不冲突的。

首先，vector 和 deque 是用来存储元素的容器，而堆是一种数据结构，其本身无法存储数据，只能依附于某个存储介质，辅助其组织数据存储的先后次序。

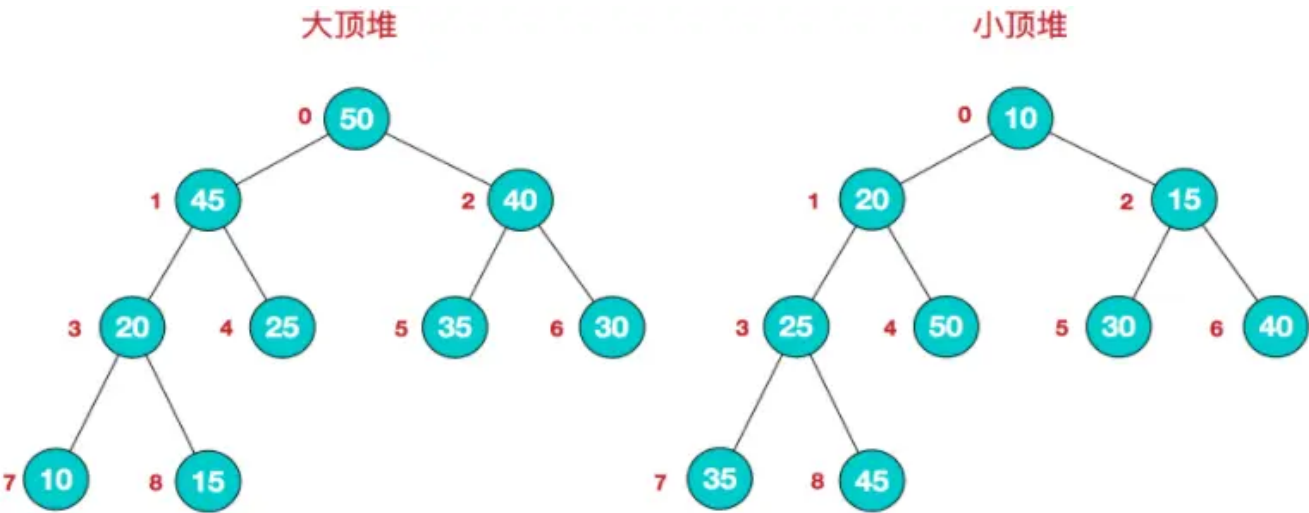
其次，priority_queue 底层采用 vector 或者 deque 作为基础容器，这毋庸置疑。但由于

vector 或 deque 容器并没有提供实现 priority_queue 容器适配器 “First in,Largest out” 特性的功能，因此 STL 选择使用堆来重新组织 vector 或 deque 容器中存储的数据，从而实现该特性。

简单的理解堆，它在是完全二叉树的基础上，要求树中所有的父节点和子节点之间，都要满足既定的排序规则：

- 如果排序规则为从大到小排序，则表示堆的完全二叉树中，每个父节点的值都要不小于子节点的值，这种堆通常称为大顶堆；
- 如果排序规则为从小到大排序，则表示堆的完全二叉树中，每个父节点的值都要不大于子节点的值，这种堆通常称为小顶堆；

下图展示了一个由 {10,20,15,30,40,25,35,50,45} 这些元素构成的大顶堆和小顶堆。其中经大顶堆组织后的数据先后次序变为 {50,45,40,20,25,35,30,10,15}，而经小顶堆组织后的数据次序为{10,20,15,25,50,30,40,35,45}。



可以看到，大顶堆中，每个父节点的值都不小于子节点；同样在小顶堆中，每个父节点的值都不大于子节点。但需要注意的是，无论是大顶堆还是小顶堆，同一父节点下子节点的次序是不做规定的，这也是经大顶堆或小顶堆组织后的数据整体依然无序的原因。

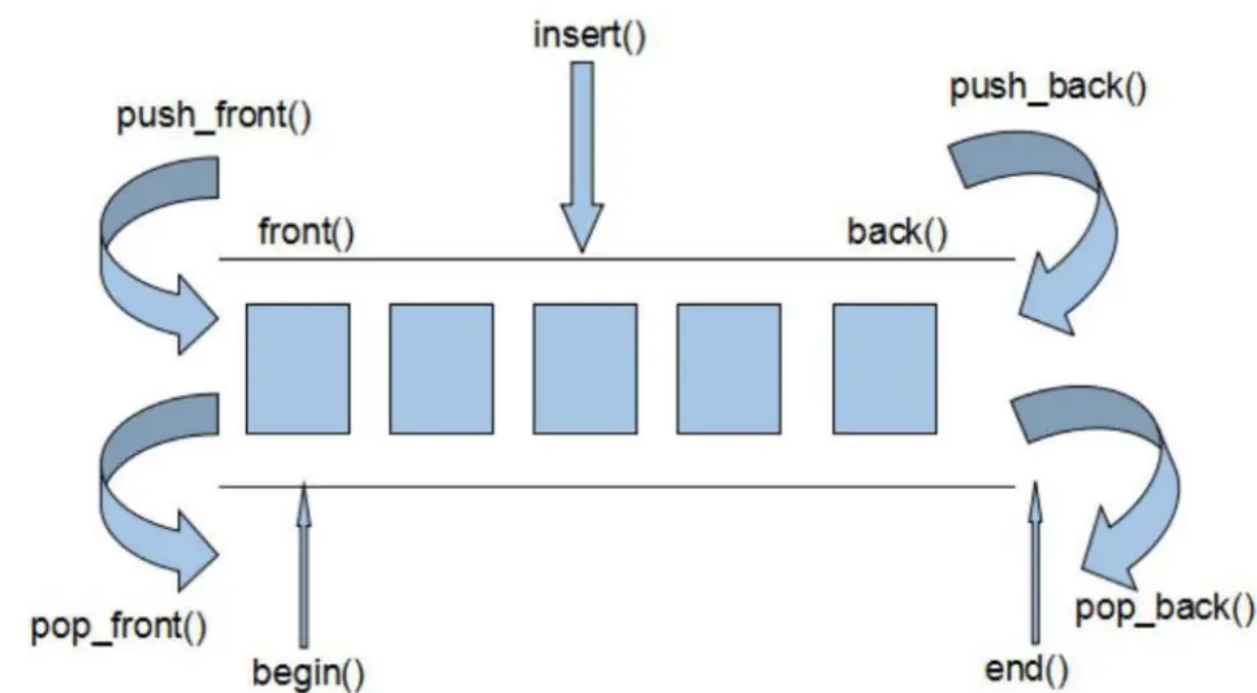
可以确定的一点是，无论是通过大顶堆或者小顶堆，总可以筛选出最大或最小的那个元素（优先级最大），并将其移至序列的开头，此功能也正是 priority_queue 容器适配器所需要的。

双向队列底层是如何实现的？

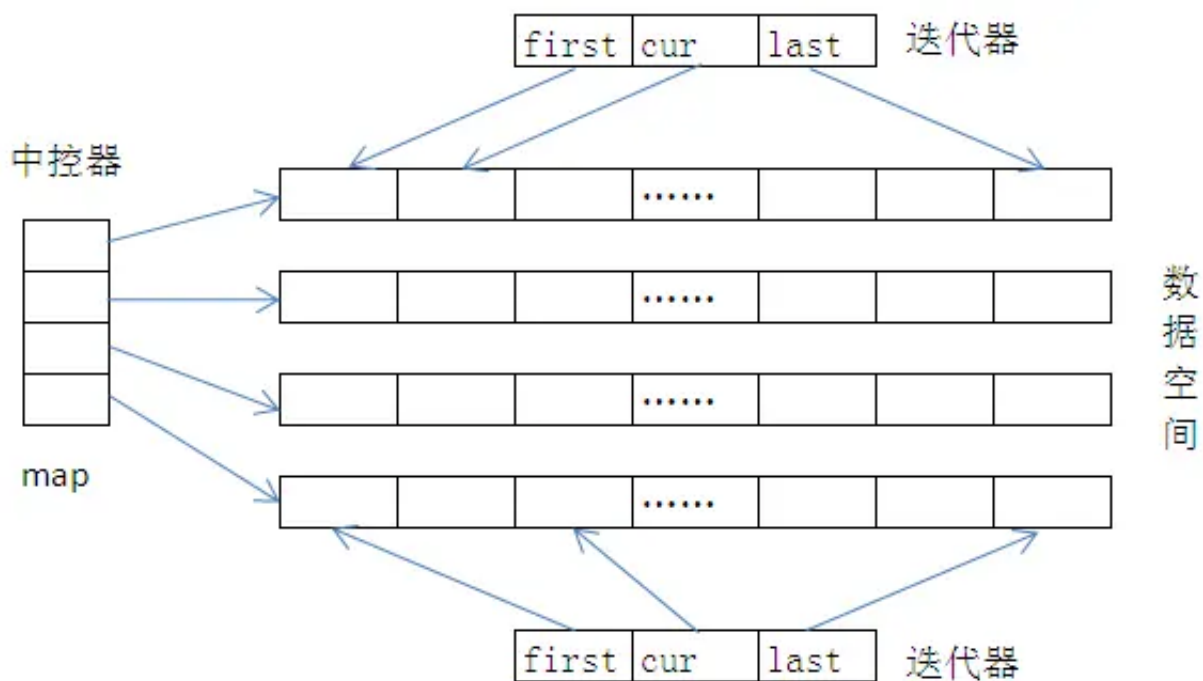
vector底层采用的是一个数组来实现，list底层采用的是一个环形的双向链表实现，而 deque则采用的是两者相结合，所谓结合，并不是两种数据结构的结合，而是某些性能上的

结合。我们知道， `vector`支持随机访问，而`list`支持常量时间的删除，`deque`支持的是随机访问以及首尾元素的插入删除。

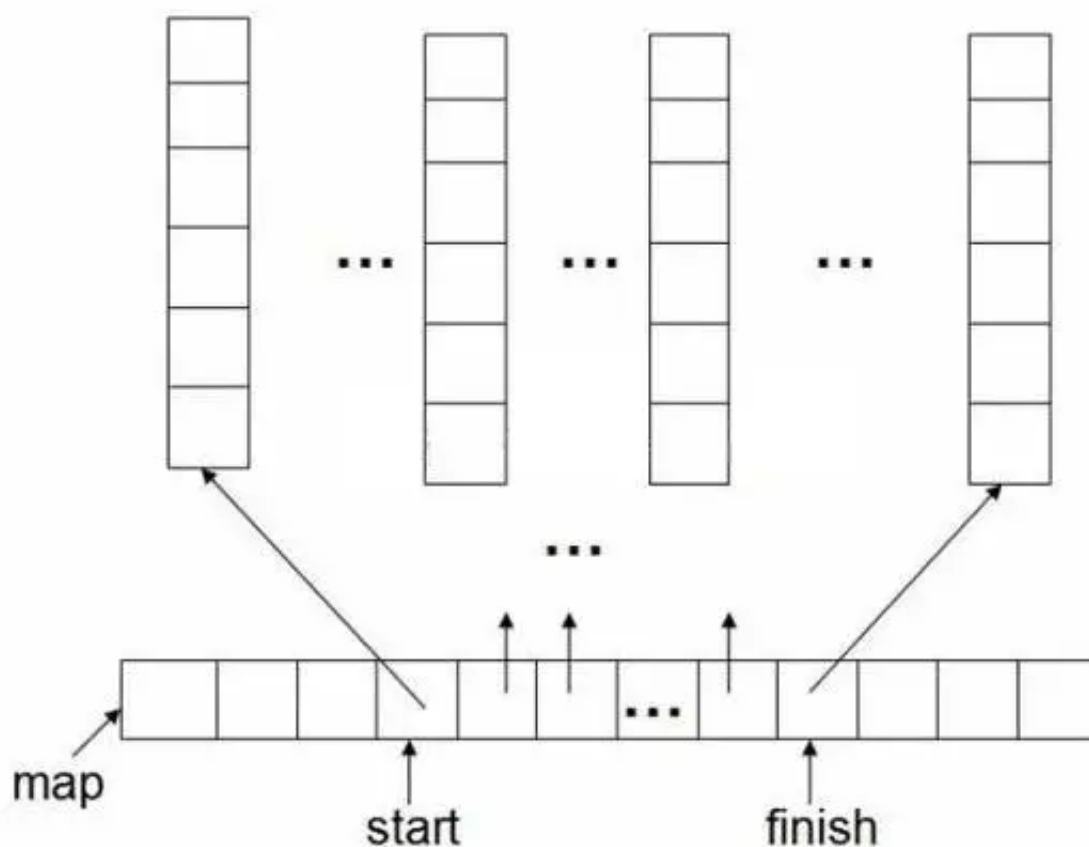
`deque`与`vector`的最大差异，一是`deque`允许常数时间内对起头端进行元素的插入和删除，二是`deque`没有所谓的容量概念，因为它是以分段连续空间组合而成，随时可以增加一段新的空间并连接起来。换句话说，像`vector`那样因旧空间不足而重新配置更大空间，复制元素，释放原空间的事情，`deque`是不会出现的。也因此，`deque`不需要提供所谓的空间保留。



双端队列`deque`是一种双向开口的存储空间分段连续的数据结构，每段数据空间内部是连续的，而每段数据空间之间则不一定连续，如下图所示：



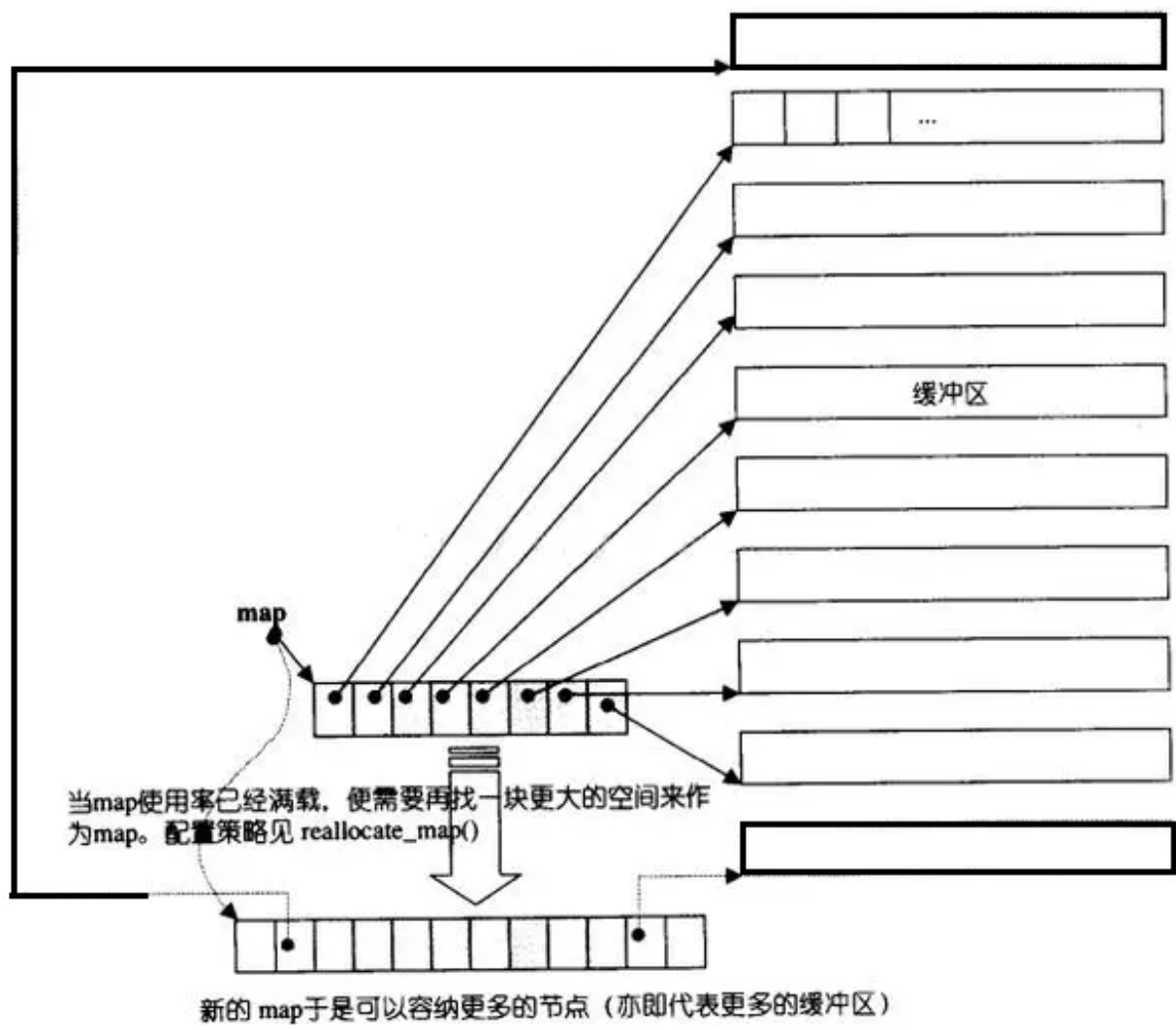
上图的deque有四段数据空间，这些空间都是程序运行过程中在堆上动态分配的。控制器（或叫map）保存着一组指针，每个指针指向一段数据空间的起始位置，通过控制器可以找到所有的数据空间。如果控制器的数据空间满了，会重新申请一块更大的空间，并将控制器的所有指针拷贝到新空间中。



如上图所示，deque是由一段一段的定量连续空间构造。一旦有必要在deque的前端或尾端

增加新空间，便配置一段连续空间，串接在整个deque的头部或尾部。

deque采用一块所谓的map（不是STL的map容器）作为主控，这里的map也是一块连续空间，其中每个元素为一个节点（也是deque的迭代器），指向另一段较大的连续线性空间，称为缓冲区，缓冲区才是deque的储存空间主体。



Std::sort的底层是怎么实现的？

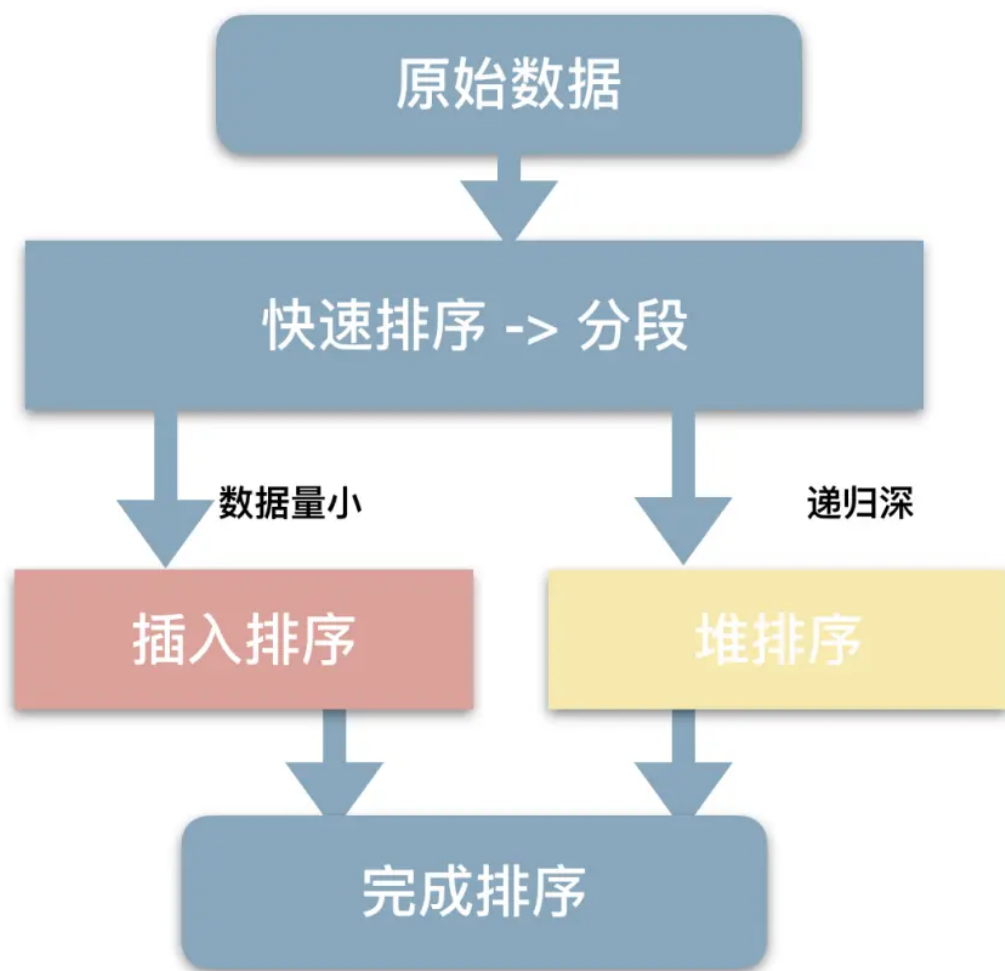
std::sort主要是三种算法的结合体：插入排序，快速排序，堆排序。

算法	时间复杂度	优点	缺点
插入排序	$O(N^2)$	当数据量很少时，效率比较高。	当数据量比较大时，时间复杂度比较高。
快速	平均 $O(N \log x N)$	大部分时候性能比较	算法时间复杂度不稳定，数据量大时递归深度很大，影响程序工

排序	，最坏 $O(N \times N)$	好。	作效率。
堆排序	$O(N \times \log N)$	算法时间复杂度稳定，比较小，适合数据量比较大的排序。	堆排序在建堆和调整堆的过程中会产生比较大的开销，数据量少的时候不适用。

std::sort 根据上文提到的几种算法的优缺点，对排序算法进行整合。

- 1. 快速排序，递归排序到一定深度后，数据已经被分为多个子区域，子区域里面的数据可能是无序的，但是子区域之间已经是有序了。
- 2. 在这多个子区域里，如果某个子区域数据个数大于阈值（16），采用堆排序，使得某个子区域内部有序。
- 3. 剩下的没有被堆排序的小区域，数据量都是小于阈值的，最后整个数据区域采用插入排序。



- 1. std::sort 采用的是分治思维，先采用快速排序，将整个区域分成多个子区域，每个子区域内部根据数据量采用不同算法。
- 2. 分治后，各个子区域局部有序后再通过整个区域进行排序。

C++智能指针

介绍一下智能指针？

在C++中，有三种常用的智能指针：`std::unique_ptr`、`std::shared_ptr`和`std::weak_ptr`。

- `std::unique_ptr`：`std::unique_ptr`是一种独占所有权的智能指针。它通过使用独占所有权的方式来管理资源，只能有一个`std::unique_ptr`指向同一个对象或数组。当`std::unique_ptr`超出作用域或被显式释放时，它会自动删除所管理的对象或数组。它通常用于表示独占的资源所有权，如动态分配的单个对象或数组。
- `std::shared_ptr`：`std::shared_ptr`是一种共享所有权的智能指针。它可以有多个`std::shared_ptr`指向同一个对象，通过引用计数来管理资源的生命周期。只有当最后一个`std::shared_ptr`超出作用域或被显式释放时，资源才会被删除。`std::shared_ptr`允许多个指针共享对同一资源的访问，通常用于表示共享的资源所有权。
- `std::weak_ptr`：`std::weak_ptr`是一种弱引用的智能指针。它可以指向由`std::shared_ptr`管理的对象，但不会增加引用计数。`std::weak_ptr`主要用于解决`std::shared_ptr`的循环引用问题，通过`std::weak_ptr.lock()`方法可以获取一个有效的`std::shared_ptr`来访问被管理的对象。

在哪些场景下会应用智能指针？

我自己是在在动态内存管理中，使用智能指针可以避免手动管理内存的麻烦和出错风险。

`shared_ptr`的作用是什么？

在传统的 C++ 编程中，使用 `new` 操作符分配的内存需要手动使用 `delete` 操作符释放，若忘记释放或者在异常情况下无法执行 `delete` 操作，就会造成内存泄漏。

`std::shared_ptr` 可以自动处理内存的释放，当不再有 `std::shared_ptr` 指向该对象时，对象的内存会被自动释放。

`std::shared_ptr` 支持多个 `std::shared_ptr` 实例共享同一个对象的所有权。它通过引用计数机制来实现这一点，每个 `std::shared_ptr` 都会维护一个引用计数，记录有多少个 `std::shared_ptr` 共享同一个对象。当引用计数变为 0 时，对象的内存会被释放。

```
#include <iostream>
#include <memory>

class MyClass {
public:
    MyClass() { std::cout << "MyClass constructor" << std::endl; }
    ~MyClass() { std::cout << "MyClass destructor" << std::endl; }
};
```

[解 Redis](#)
[面试八股题](#)
[🔥 大模型项目 🔥](#)
[🌟 求职训练营 🌟](#)
[🌟 AI面试 🌟](#)
[大厂面试真题](#)
[其他](#)

```
std::shared_ptr<MyClass> ptr2 = ptr1; // 共享所有权
std::cout << "Use count: " << ptr1.use_count() << std::endl; // 输出
ptr2.reset(); // 释放 ptr2 的所有权
std::cout << "Use count: " << ptr1.use_count() << std::endl; // 输出
// 当 ptr1 离开作用域时, MyClass 对象的内存会被释放
return 0;
}
```

在这个例子中, `ptr1` 和 `ptr2` 共享同一个 `MyClass` 对象的所有权, 引用计数变为 2。当调用 `ptr2.reset()` 时, `ptr2` 释放了对对象的所有权, 引用计数减为 1。当 `ptr1` 离开作用域时, 引用计数变为 0, 对象的内存被释放。

weak_ptr的作用是什么? 如何和shared_ptr结合使用?

`std::weak_ptr` 主要用于辅助 `std::shared_ptr` 进行内存管理, 解决 `std::shared_ptr` 可能存在的循环引用问题, 同时还可以用于观察 `std::shared_ptr` 所管理对象的生命周期。

当两个或多个 `std::shared_ptr` 相互引用形成循环时, 会导致引用计数永远不会降为 0, 从而造成内存泄漏。 `std::weak_ptr` 不会增加所指向对象的引用计数, 因此可以打破这种循环引用。

```
#include <iostream>
#include <memory>

class B;

class A {
```



目录



侧边栏



夜间



技术群



做项目



训练营



AI面试



面试鸭



校招汇总



资料



赞赏我

[上一篇](#)[下一篇](#)

```
public:
    std::shared_ptr<B> b_ptr;
    ~A() { std::cout << "A destructor" << std::endl; }
};

class B {
public:
    std::weak_ptr<A> a_ptr; // 使用 std::weak_ptr 打破循环引用
    ~B() { std::cout << "B destructor" << std::endl; }
};

int main() {
    std::shared_ptr<A> a = std::make_shared<A>();
    std::shared_ptr<B> b = std::make_shared<B>();
    a->b_ptr = b;
    b->a_ptr = a;
    return 0;
}
```

在上述代码中，如果 B 类中的 a_ptr 也使用 std::shared_ptr，就会形成循环引用，导致 A 和 B 对象的内存无法释放。使用 std::weak_ptr 后，b->a_ptr 不会增加 A 对象的引用计数，当 main 函数结束时，a 和 b 的引用计数降为 0，A 和 B 对象的内存会被正确释放。

另外，std::weak_ptr 可以用于观察 std::shared_ptr 所管理对象的生命周期。通过 std::weak_ptr 的 expired() 方法可以检查所指向的对象是否已经被释放。

```
#include <iostream>
#include <memory>

int main() {
    std::shared_ptr<int> shared = std::make_shared<int>(42);
    std::weak_ptr<int> weak = shared;

    if (!weak.expired()) {
        std::cout << "Object is still alive." << std::endl;
    }

    shared.reset();
    if (weak.expired()) {
        std::cout << "Object has been destroyed." << std::endl;
    }
}
```

```
    return 0;  
}
```

在这个例子中，`weak` 观察 `shared` 所管理的对象。当 `shared` 释放对象后，`weak.expired()` 返回 `true`，表示对象已经被销毁。

智能指针会造成内存泄漏吗？

会的，比如循环引用场景，在多个 `shared_ptr` 形成循环引用，资源将无法释放。

`std::shared_ptr`：多个 `shared_ptr` 对象可以共享同一个资源的所有权，它们会维护一个引用计数。只有当引用计数为零时，才会释放资源。这种智能指针的内存管理是自动的，因此可以帮助我们避免显式地释放内存或出现内存泄漏的情况。

例如：

```
std::shared_ptr<int> sharedPtr1 = std::make_shared<int>(10);  
std::shared_ptr<int> sharedPtr2 = sharedPtr1;  
sharedPtr1.reset(); // 不会导致内存泄漏，资源仍由sharedPtr2管理  
sharedPtr2.reset(); // 资源被释放
```

在使用 `std::shared_ptr` 时，需要注意循环引用的情况。如果两个或多个 `shared_ptr` 对象相互引用，形成循环依赖，它们的引用计数永远不会达到零，资源将无法释放，从而导致内存泄漏。

针对这个问题，可以使用 `weak_ptr` 弱引用来解决这个问题。`weak_ptr` 是用来监视 `shared_ptr` 的生命周期，它不管理 `shared_ptr` 内部的指针，它的拷贝的析构都不会影响引用计数，纯粹是作为一个旁观者监视 `shared_ptr` 中管理的资源是否存在，可以用来返回 `this` 指针和解决循环引用问题。

一个 `unique_ptr` 怎么赋值给另一个 `unique_ptr` 对象？

借助 `std::move()` 可以实现将一个 `unique_ptr` 对象赋值给另一个 `unique_ptr` 对象，其目的是实现所有权的转移。也可以让当前 `unique_ptr` 调用 `release` 释放对 `ptr` 控制权，然后在另一个 `unique_ptr` 获取控制权。

```
// A 作为一个类
std::unique_ptr<A> ptr1(new A());
std::unique_ptr<A> ptr2 = std::move(ptr1);
std::unique_ptr<A> ptr3(ptr2.release());
```

使用智能指针会出现什么问题？ 怎么解决？

智能指针可能出现的问题：循环引用

在如下例子中定义了两个类 Parent、Child，在两个类中分别定义另一个类的对象的共享指针，由于在程序结束后，两个指针相互指向对方的内存空间，导致内存无法释放。

```
#include <iostream>
#include <memory>

using namespace std;

class Child;
class Parent;

class Parent {
private:
    shared_ptr<Child> ChildPtr;
public:
    void setChild(shared_ptr<Child> child) {
        this->ChildPtr = child;
    }

    void doSomething() {
        if (this->ChildPtr.use_count()) {

        }
    }

    ~Parent() {
    }
};

class Child {
private:
```

```

        shared_ptr<Parent> ParentPtr;
public:
    void setParent(shared_ptr<Parent> parent) {
        this->ParentPtr = parent;
    }
    void doSomething() {
        if (this->ParentPtr.use_count()) {

        }
    }
    ~Child() {
    }
};

int main() {
    weak_ptr<Parent> wpp;
    weak_ptr<Child> wpc;
    {
        shared_ptr<Parent> p(new Parent);
        shared_ptr<Child> c(new Child);
        p->setChild(c);
        c->setParent(p);
        wpp = p;
        wpc = c;
        cout << p.use_count() << endl; // 2
        cout << c.use_count() << endl; // 2
    }
    cout << wpp.use_count() << endl; // 1
    cout << wpc.use_count() << endl; // 1
    return 0;
}

```

循环引用的解决方法： weak_ptr

循环引用：该被调用的析构函数没有被调用，从而出现了内存泄漏。

- weak_ptr 对被 shared_ptr 管理的对象存在非拥有性（弱）引用，在访问所引用的对象前必须先转化为 shared_ptr；
- weak_ptr 用来打断 shared_ptr 所管理对象的循环引用问题，若这种环被孤立（没有指向环中的外部共享指针），shared_ptr 引用计数无法抵达 0，内存被泄露；令环中的指针之一为弱指针可以避免该情况；
- weak_ptr 用来表达临时所有权的概念，当某个对象只有存在时才需要被访问，而且随时

可能被他人删除，可以用 weak_ptr 跟踪该对象；需要获得所有权时将其转化为 shared_ptr，此时如果原来的 shared_ptr 被销毁，则该对象的生命期被延长至这个临时的 shared_ptr 同样被销毁。

```
#include <iostream>
#include <memory>

using namespace std;

class Child;
class Parent;

class Parent {
private:
    //shared_ptr<Child> ChildPtr;
    weak_ptr<Child> ChildPtr;
public:
    void setChild(shared_ptr<Child> child) {
        this->ChildPtr = child;
    }

    void doSomething() {
        //new shared_ptr
        if (this->ChildPtr.lock()) {

        }
    }

    ~Parent() {
    }
};

class Child {
private:
    shared_ptr<Parent> ParentPtr;
public:
    void setParent(shared_ptr<Parent> parent) {
        this->ParentPtr = parent;
    }

    void doSomething() {
        if (this->ParentPtr.use_count()) {

        }
    }
}
```

```

    }
    ~Child() {
    }
};

int main() {
    weak_ptr<Parent> wpp;
    weak_ptr<Child> wpc;
    {
        shared_ptr<Parent> p(new Parent);
        shared_ptr<Child> c(new Child);
        p->setChild(c);
        c->setParent(p);
        wpp = p;
        wpc = c;
        cout << p.use_count() << endl; // 2
        cout << c.use_count() << endl; // 1
    }
    cout << wpp.use_count() << endl; // 0
    cout << wpc.use_count() << endl; // 0
    return 0;
}

```

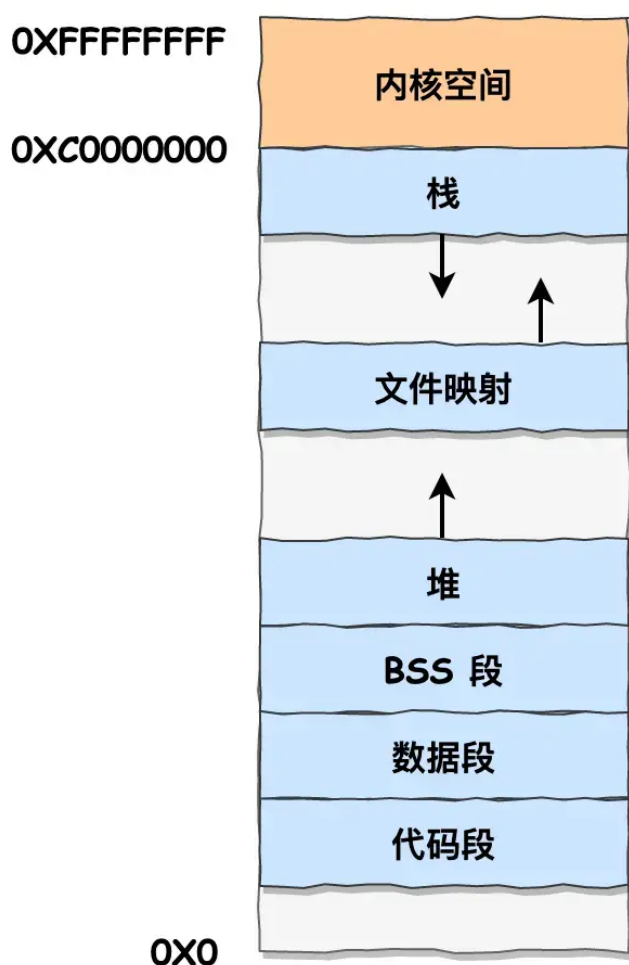
C++内存管理

虚拟内存介绍一下

- 第一，虚拟内存可以使得进程对运行内存超过物理内存大小，因为程序运行符合局部性原理，CPU 访问内存会有很明显的重复访问的倾向性，对于那些没有被经常使用到的内存，我们可以把它换出到物理内存之外，比如硬盘上的 swap 区域。
- 第二，由于每个进程都有自己的页表，所以每个进程的虚拟内存空间就是相互独立的。进程也没有办法访问其他进程的页表，所以这些页表是私有的，这就解决了多进程之间地址冲突的问题。
- 第三，页表里的页表项中除了物理地址之外，还有一些标记属性的比特，比如控制一个页的读写权限，标记该页是否存在等。在内存访问方面，操作系统提供了更好的安全性。

虚拟内存用户态的地址空间怎么分配的

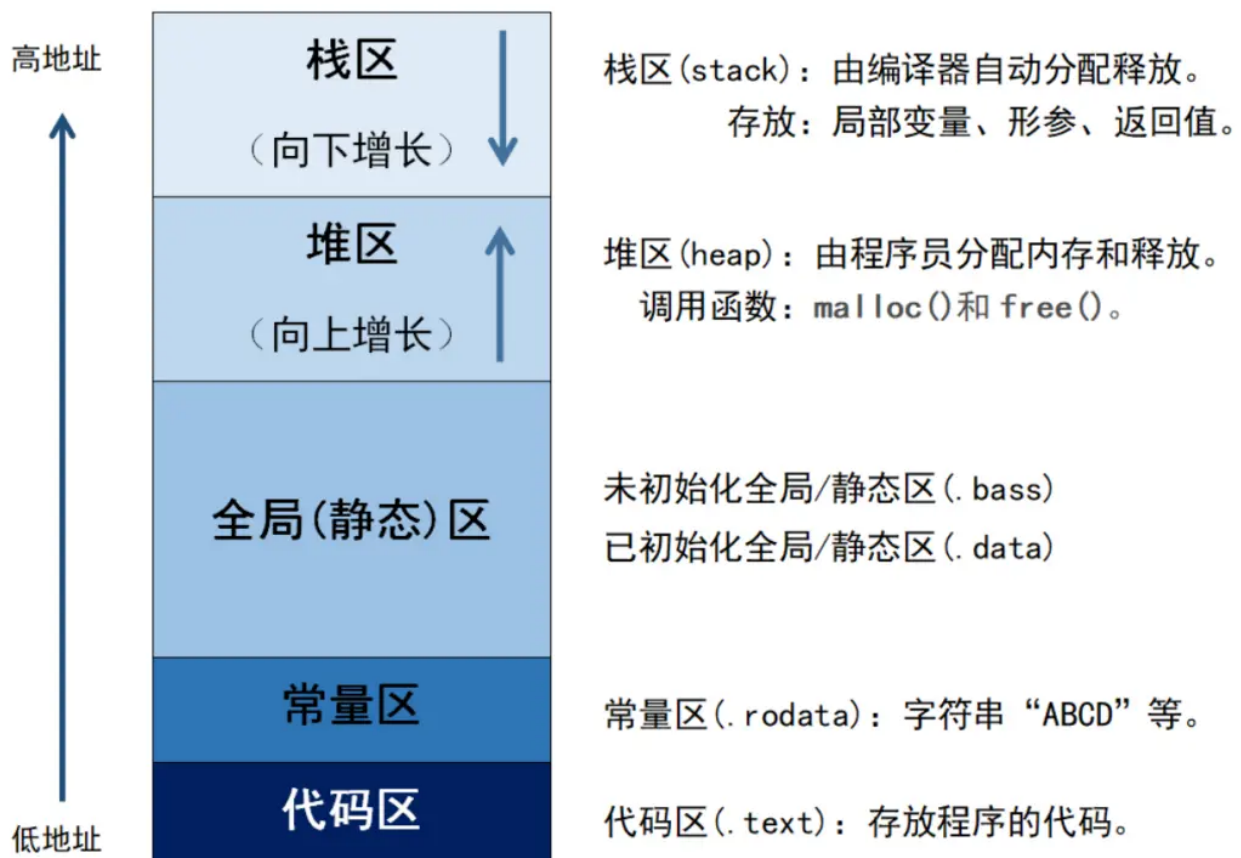
用户态地址空间是一块连续的虚拟地址范围（比如 32 位系统通常是 0~3GB，64 位系统范围更大），按“从低到高”分成几块，每块有专门用途，互不干扰。



- 代码段，包括二进制可执行代码；
- 数据段，包括已初始化的静态常量和全局变量；
- BSS 段，包括未初始化的静态变量和全局变量；
- 堆段，包括动态分配的内存，从低地址开始向上增长；
- 文件映射段，包括动态库、共享内存等
- 栈段，包括局部变量和函数调用的上下文等，比如函数里定义的 `int x = 5`，就在栈里。栈的大小是固定的，一般是 8MB。当然系统也提供了参数，以便我们自定义大小；

C++中的内存分区有哪些？

在C++中，内存主要分为以下五个区域：



- 栈区 (Stack)：由编译器自动分配释放，存放函数的参数值，局部变量等。其操作方式类似于数据结构中的栈。
- 堆区 (Heap)：一般由程序员分配释放，若程序员不释放，程序结束时可能由OS回收。注意，与数据结构中的堆是两回事，分配方式倒是类似于链表。
- 全局区 (静态区) (Static)：全局变量和静态变量被分配到同一块内存中。在C++中，全局区还包含了常量区，字符串常量和和其他常量也是存储在此。
- 常量区：是全局区的一部分，存放常量，不允许修改。
- 代码区 (Text)：存放函数体的二进制代码。

介绍一下内存对齐

内存对齐就是就是将数据存放在内存的某个位置，使得CPU可以更快地访问到这个数据，以空间换时间的方式来提高 cpu 访问数据的性能。

在C++中，内存对齐主要涉及到两个概念：对齐边界和填充字节。

- 对齐边界：一般情况下，编译器会自动地将数据存放在它的自然边界上。例如，`int`类型的数据，它的大小为4字节，编译器会将其存放在4的倍数的地址上。这就是所谓的对齐边界。
- 填充字节：为了满足对齐边界的要求，编译器有时候需要在数据之间填充一些字节。这

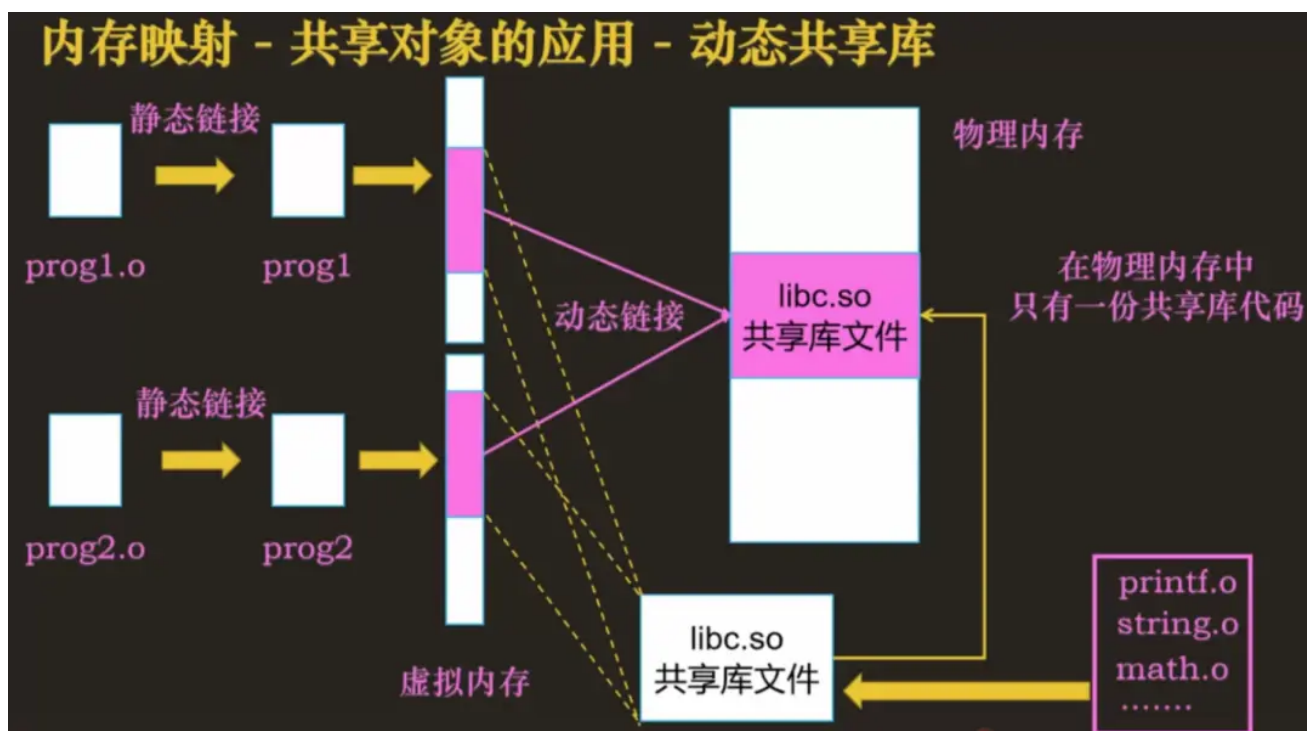
些字节没有实际的意义，只是为了满足内存对齐的要求。

为什么要字节对齐？

- 平台原因(移植原因)：不是所有的硬件平台都能访问任意地址上的任意数据的；某些硬件平台只能在某些地址处取某些特定类型的数据，否则抛出硬件异常。
- 性能原因：数据结构(尤其是栈)应该尽可能地在自然边界上对齐。原因在于，为了访问未对齐的内存，处理器需要作两次内存访问；而对齐的内存访问仅需要一次访问。

动态链接库怎么装载到内存的？

通过用mmap把该库直接映射到各个进程的地址空间中，尽管每个进程都认为自己地址空间中加载了该库，但实际上该库在内存中只有一份，mmap就这样很神奇和动态链接库联动起来了。



函数调用的时候压栈怎么样的

函数调用时，会进行以下压栈操作：

- 保存返回地址：在函数调用前，调用指令会将下一条指令的地址（即函数调用后需要继续执行的地址）压入栈中，以便函数执行完毕后能够正确返回到调用点。
- 保存调用者的栈帧指针：在函数调用前，调用指令会将当前栈帧指针（即调用者的栈指针）压入栈中，以便函数执行完毕后能够恢复到调用者的执行状态。

- 传递参数：函数调用时，会将参数值依次压入栈中，这些参数值在函数内部可以通过栈来访问。
- 分配局部变量空间：函数调用时，会为局部变量分配空间，这些局部变量会被保存在栈中。栈指针会相应地移动以适应新的局部变量空间。

C++中堆和栈的区别

申请方式：

- 栈：由系统自动分配。例如，声明在函数中一个局部变量 `int b`，系统自动在栈中为b开辟空间。
- 堆：需要程序员自己申请，并指明大小，在C语言中通过malloc函数，如 `p1 = (char *)malloc(10);`，在C++中用new运算符，如 `p2 = new char[20]`

申请后系统的响应：

- 栈：只要栈的剩余空间大于所申请空间，系统将为程序提供内存，否则将报异常提示栈溢出。
- 堆：首先应该知道操作系统有一个记录空闲内存地址的链表，当系统收到程序的申请时，会遍历该链表，寻找第一个空间大于所申请空间的堆结点，然后将该结点从空闲结点链表中删除，并将该结点的空间分配给程序，另外，对于大多数系统，会在这块内存空间中的首地址处记录本次分配的大小，这样，代码中的delete语句才能正确的释放本内存空间。另外，由于找到的堆结点的大小不一定正好等于申请的大小，系统会自动的将多余的那部分重新放入空闲链表中。

申请大小的限制：

- 栈：栈是向低地址扩展的数据结构，是一块连续的内存的区域。这句话的意思是栈顶的地址和栈的最大容量是系统预先规定好的，操作系统中，栈的大小是几MB，如果申请的空间超过栈的剩余空间时，将提示overflow。因此，能从栈获得的空间较小。
- 堆：堆是向高地址扩展的数据结构，是不连续的内存区域。这是由于系统是用链表来存储的空闲内存地址的，自然是不连续的，而链表的遍历方向是由低地址向高地址。堆的大小受限于计算机系统中有效的虚拟内存。由此可见，堆获得的空间比较灵活，也比较大。

生命周期：

- 栈：栈的内存管理是自动的，变量的内存会在其作用域结束时自动释放

- 堆：堆的内存管理需要手动进行，需要使用new关键字分配内存，并使用delete或delete[]关键字释放内存，否则会导致内存泄漏。

new是在内存上哪一块去分配的内存？

new所申请的内存区域在C++中称为自由存储区。很多编译器的new/delete都是以malloc/free为基础来实现的，所以通常都是借由堆实现来实现自由存储，这时候就可以说new所申请的内存区域在堆上。

如果new内存失败了会怎么样？

会抛出std::bad_alloc异常，如果加上std::nothrow关键字，A* p = new (std::nothrow) A;，new 就不会抛出异常而是会返回空指针。

C++ malloc和new的区别是什么？

new和malloc区别：

- **分配内存的位置**：malloc是从堆上动态分配内存，new是从自由存储区为对象动态分配内存。自由存储区的位置取决于operator new的实现。自由存储区不仅可以为堆，还可以是静态存储区，这都看operator new在哪里为对象分配内存。
- **返回类型安全性**：malloc内存分配成功后返回void*，然后再强制类型转换为需要的类型；new操作符分配内存成功后返回与对象类型相匹配的指针类型；因此new是符合类型安全的操作符。
- **内存分配失败返回值**：malloc内存分配失败后返回NULL。new分配内存失败则会抛异常（bad_alloc）。
- **分配内存的大小的计算**：使用new操作符申请内存分配时无须指定内存块的大小，编译器会根据类型信息自行计算，而malloc则需要显式地指出所需内存的尺寸。
- **是否可以被重载**：operator new /operator delete可以被重载。而malloc/free则不能重载。

new 和 malloc 如何判断是否申请到内存？

- malloc：成功申请到内存，返回指向该内存的指针；分配失败，返回 NULL 指针。
- new：内存分配成功，返回该对象类型的指针；分配失败，抛出 bad_alloc 异常。

delete和free的区别

- new/delete是C++的操作符，而malloc/free是C中的函数。
- new做两件事，一是分配内存，二是调用类的构造函数；同样，delete会调用类的析构函数和释放内存。而malloc和free只是分配和释放内存。
- new建立的是一个对象，而malloc分配的是一块内存；new建立的对象可以用成员函数访问，不要直接访问它的地址空间；malloc分配的是一块内存区域，用指针访问，可以在里面移动指针；new出来的指针是带有类型信息的，而malloc返回的是void指针。
- new/delete是保留字，不需要头文件支持；malloc/free需要头文件库函数支持。

delete 实现原理？ delete 和 delete[] 的区别？

delete 的实现原理：

- 首先执行该对象所属类的析构函数；
- 进而通过调用 operator delete 的标准库函数来释放所占的内存空间。

delete 和 delete [] 的区别：

- delete 用来释放单个对象所占的空间，只会调用一次析构函数；
- delete [] 用来释放数组空间，会对数组中的每个成员都调用一次析构函数。

malloc 的原理？ malloc 的底层实现？

malloc 的原理：

- 当开辟的空间小于 128K 时，调用 brk() 函数，通过移动 _enddata 来实现；
- 当开辟空间大于 128K 时，调用 mmap() 函数，通过在虚拟地址空间中开辟一块内存空间来实现。

malloc 的底层实现：

- brk() 函数实现原理：向高地址的方向移动指向数据段的高地址的指针 _enddata。
- mmap 内存映射原理：
 1. 进程启动映射过程，并在虚拟地址空间中为映射创建虚拟映射区域；
 2. 调用内核空间的系统调用函数 mmap()，实现文件物理地址和进程虚拟地址的一一映射关系；
 3. 进程发起对这片映射空间的访问，引发缺页异常，实现文件内容到物理内存（主存）的

拷贝。

malloc 1KB和1MB 有什么区别？

malloc() 源码里默认定义了一个阈值：

- 如果用户分配的内存小于 128 KB，则通过 brk() 申请内存；
- 如果用户分配的内存大于 128 KB，则通过 mmap() 申请内存；

注意，不同的 glibc 版本定义的阈值也是不同的。

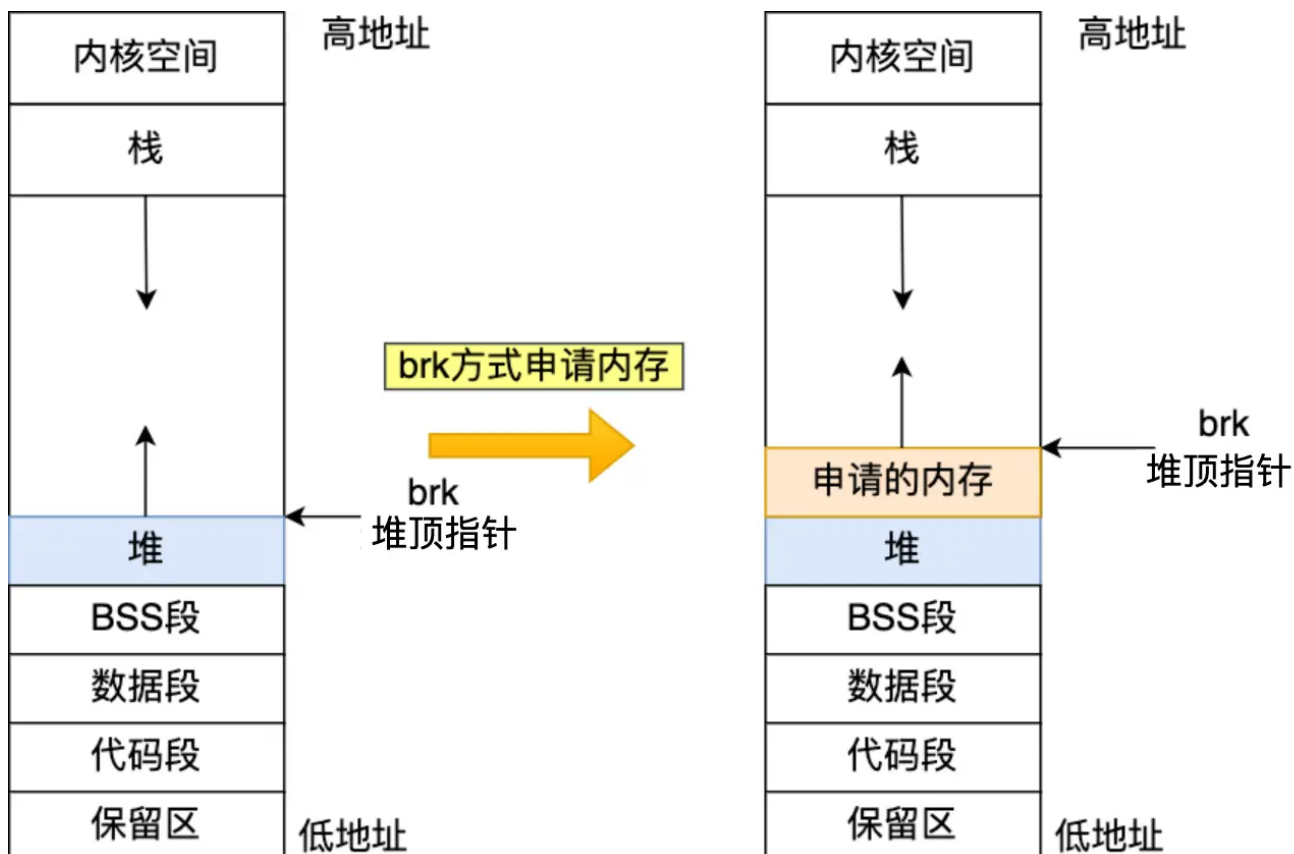
介绍一下malloc的brk, mmap

实际上，malloc() 并不是系统调用，而是 C 库里的函数，用于动态分配内存。

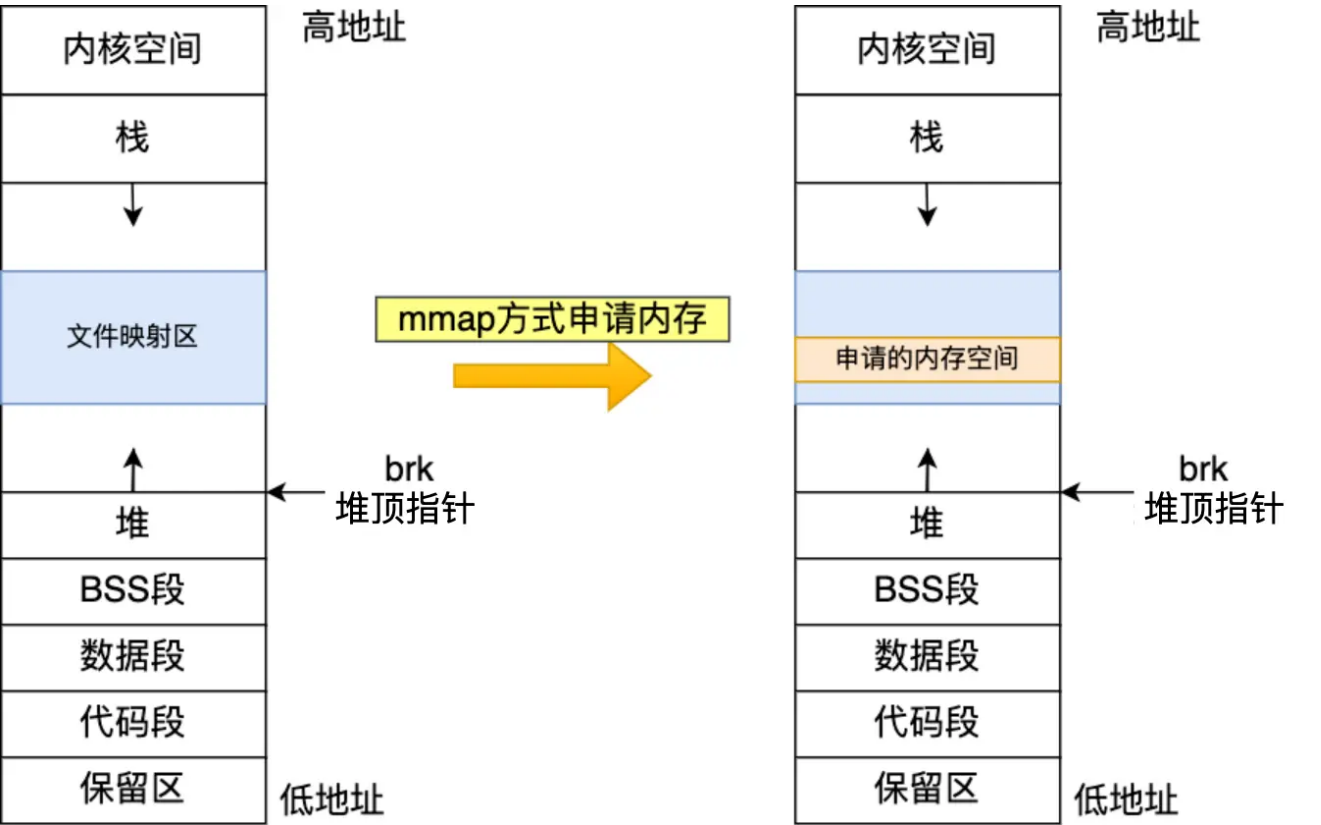
malloc 申请内存的时候，会有两种方式向操作系统申请堆内存。

- 方式一：通过 brk() 系统调用从堆分配内存
- 方式二：通过 mmap() 系统调用在文件映射区域分配内存；

方式一实现的方式很简单，就是通过 brk() 函数将「堆顶」指针向高地址移动，获得新的内存空间。如下图：



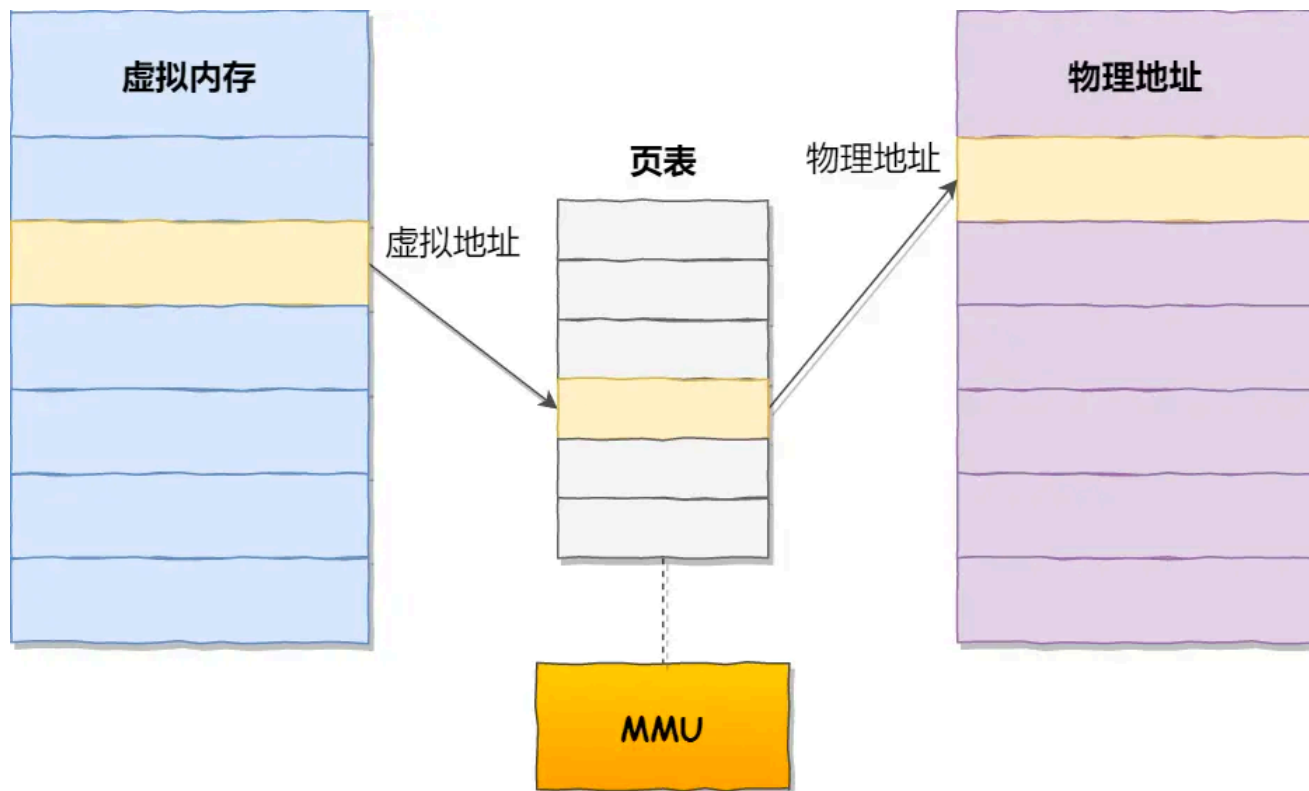
方式二通过 `mmap()` 系统调用中「私有匿名映射」的方式，在文件映射区分配一块内存，也就是从文件映射区“偷”了一块内存。如下图：



分页内存管理说一下

分页是把整个虚拟和物理内存空间切成一段段固定尺寸的大小。这样一个连续并且尺寸固定的内存空间，我们叫页（*Page*）。在 Linux 下，每一页的大小为 4KB。

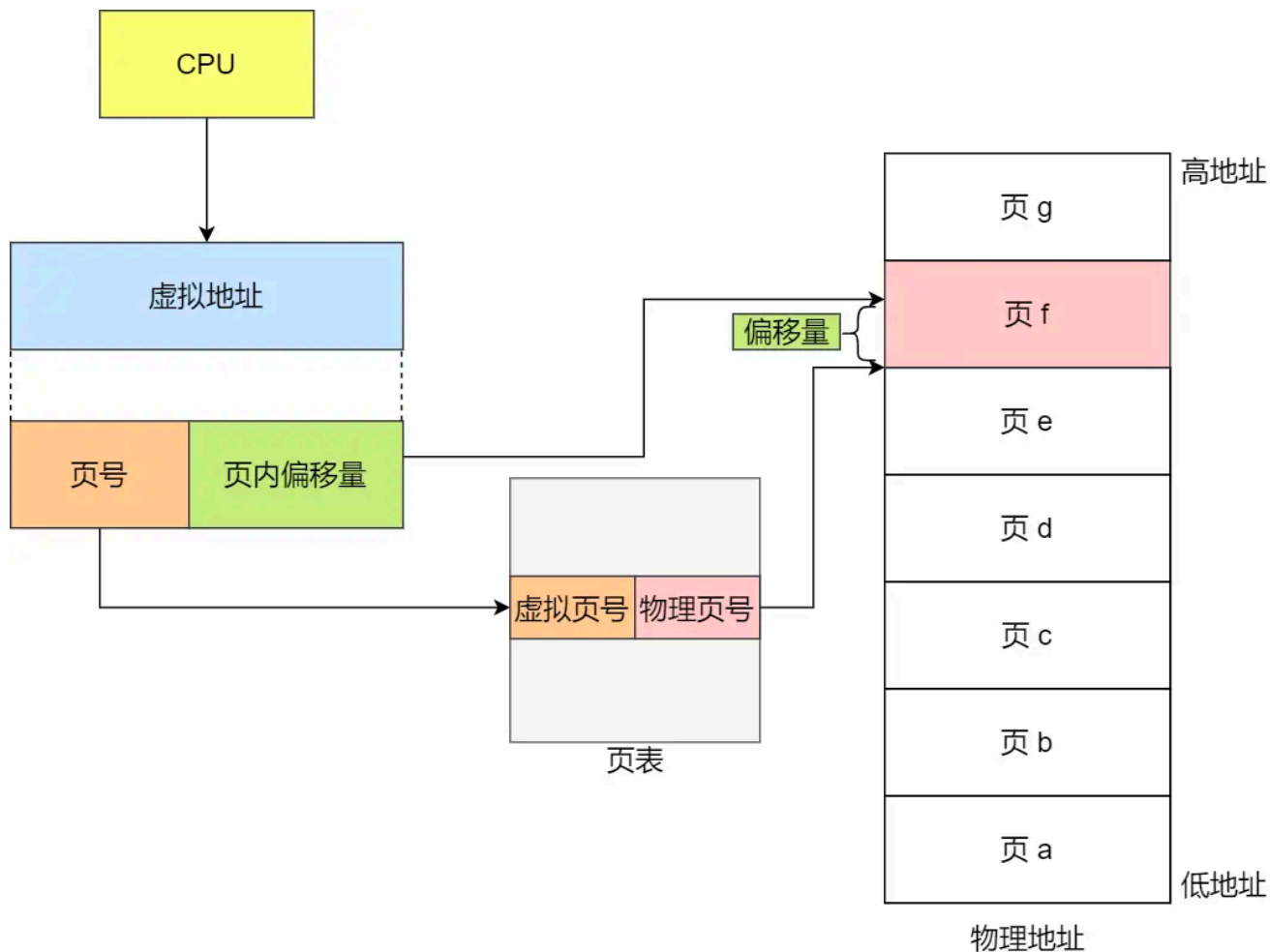
虚拟地址与物理地址之间通过页表来映射，如下图：



页表是存储在内存里的，**内存管理单元**（**MMU**）就做将虚拟内存地址转换成物理地址的工作。

而当进程访问的虚拟地址在页表中查不到时，系统会产生一个**缺页异常**，进入系统内核空间分配物理内存、更新进程页表，最后再返回用户空间，恢复进程的运行。

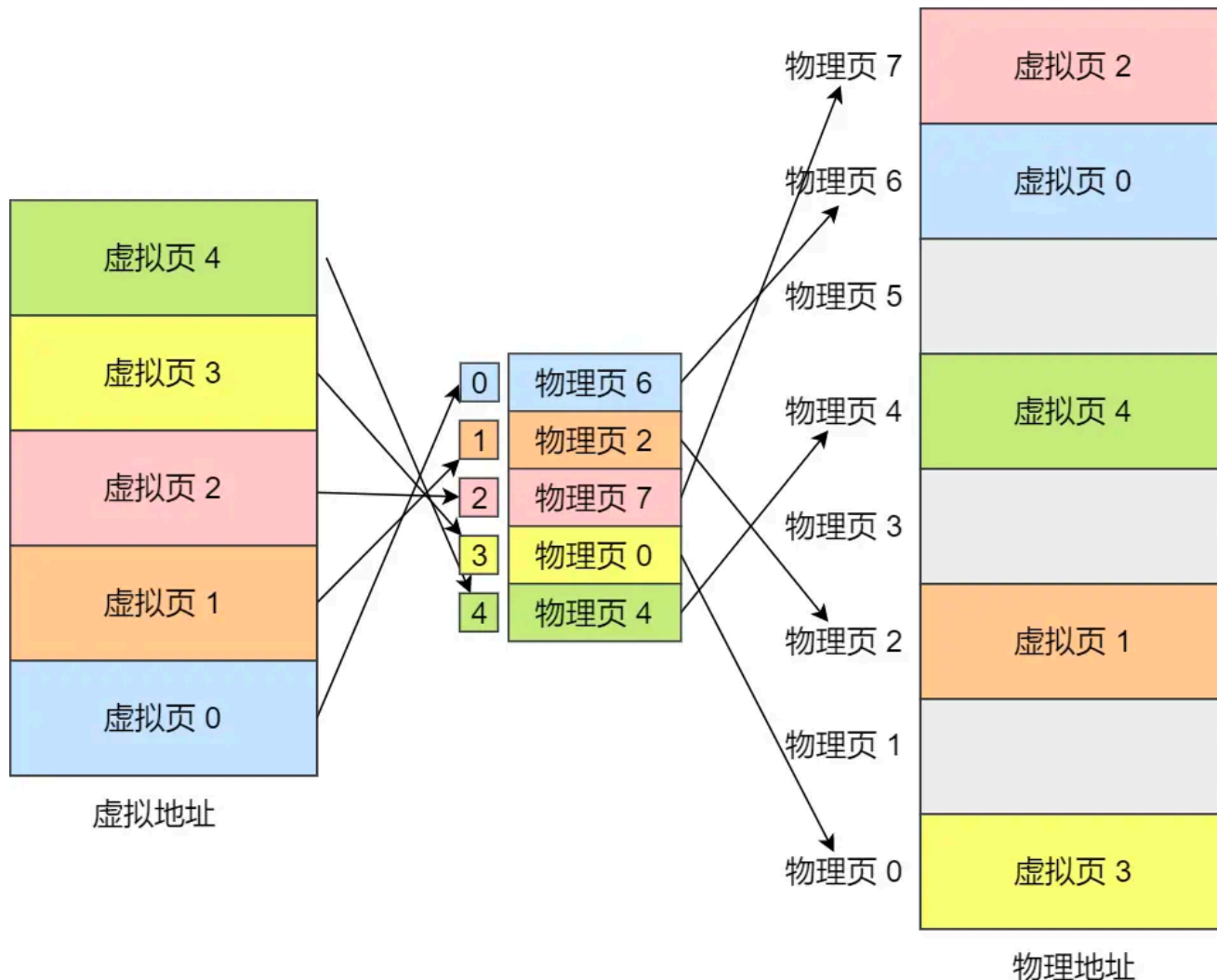
在分页机制下，虚拟地址分为两部分，**页号**和**页内偏移**。页号作为页表的索引，**页表**包含物理页每页所在**物理内存的基地址**，这个基地址与页内偏移的组合就形成了物理内存地址，见下图。



总结一下，对于一个内存地址转换，其实就是这样三个步骤：

- 把虚拟内存地址，切分成页号和偏移量；
- 根据页号，从页表里面，查询对应的物理页号；
- 直接拿物理页号，加上前面的偏移量，就得到了物理内存地址。

下面举个例子，虚拟内存中的页通过页表映射为了物理内存中的页，如下图：



C++新特性

说说你了解的C++11相关特性

- **自动类型推断 (auto)**：引入了auto关键字，可以根据变量初始化表达式的类型自动推断变量的类型，使得代码更具灵活性和可读性。
- **范围for循环**：通过for (element : container)语法，允许直接遍历容器中的每个元素，简化了迭代操作，减少了代码量。
- **移动语义和右值引用**：通过引入右值引用 (&&) 和移动构造函数，减少了资源管理时的不必要拷贝操作，提高了性能。
- **智能指针**：std::shared_ptr和std::weak_ptr等智能指针类的引入，帮助管理动态分配的内存，避免内存泄漏和悬挂指针等问题。
- **Lambda表达式**：引入了匿名函数的Lambda表达式语法，能够更方便地定义和使用函数对象，减少了冗余代码。
- **nullptr空指针**：引入了nullptr关键字，用于表示空指针，替代了传统的NULL，避免了空指针常量与整数间的模糊问题。

- **初始化列表**：通过使用花括号{}来对对象初始化，统一了初始化语法，提供了更安全、简洁的初始化方式。
- **默认和删除成员函数**：引入了=default和=delete来指明默认构造函数、拷贝构造函数等的生成和禁止。
- **强类型枚举**：引入了枚举类（enum class），解决了传统枚举类型带来的全局命名空间和类型安全问题。
- **多线程支持**：引入了std::thread、std::mutex等多线程支持库，使得并发编程更加方便和安全。
- **泛型编程优化**：引入了constexpr关键字，允许在编译时计算表达式，提高了程序的性能。

C++11 新特性了解哪些内容？

类型推导与简化语法

特性名称	描述	示例
auto 关键字	自动推导变量类型，简化复杂类型声明（如迭代器）。	auto x = 42; → int x
decltype	推导表达式类型，保留 const 和引用属性，适用于模板编程。	int i=1; decltype(i) j = i; → int j

右值引用与移动语义

右值引用 (&&)	区分左值/右值，支持资源高效转移（如临时对象）。	std::vector<int> v2 = std::move(v1);
移动构造函数/赋值运算符	减少深拷贝开销，通过资源转移提升性能。	类中定义 ClassName(ClassName&& other) noexcept;
完美转发 (std::forward)	保持参数原始类型，避免多次拷贝。	模板中使用 std::forward<T>(arg)

智能指针

std::unique_ptr	独占所有权，不可复制但可移动，替代 auto_ptr 。	std::unique_ptr<int> ptr = std::make_unique<int>(10);

<code>std::shared_ptr</code>	共享所有权，引用计数管理资源，线程安全。	<code>std::shared_ptr<int> ptr = std::make_shared<int>(10);</code>
<code>std::weak_ptr</code>	解决 <code>shared_ptr</code> 循环引用问题。	<code>std::weak_ptr<int> w_ptr = s_ptr;</code>

函数与模板增强

Lambda 表达式	匿名函数，支持捕获外部变量，简化回调和算法。	<code>std::sort(vec.begin(), vec.end(), [](int a, int b) { return a < b; });</code>
变长参数模板	支持任意数量/类型的模板参数，用于元编程和容器设计。	<code>template<typename... Args> void func(Args... args);</code>
<code>constexpr</code> 常量表达式	编译时求值，优化性能，允许函数在编译期执行。	<code>constexpr int factorial(int n) { return n <=1 ? 1 : n*factorial(n-1); }</code>

并发编程支持

<code>std::thread</code>	原生多线程支持，结合互斥锁和原子操作实现同步。	<code>std::thread t(func); t.join();</code>
<code>std::async</code> 和 <code>std::future</code>	简化异步任务管理，获取异步操作结果。	<code>auto future = std::async(func); int result = future.get();</code>

auto 类型推导的原理是什么？

auto 类型推导的原理：编译器根据初始值来推算变量的类型，要求用 auto 定义变量时必须要有初始值。编译器推断出来的 auto 类型有时和初始值类型并不完全一样，编译器会适当改变结果类型使其更符合初始化规则。

lambda表达式的原理是什么？

从本质上讲，Lambda 表达式是编译器自动生成的一个匿名的函数对象（也称为仿函数）。当我们编写一个 Lambda 表达式时，编译器会创建一个未命名的类，这个类重载了函数调用运算符 `operator()` 。

```
#include <iostream>
```

```
int main() {
    auto lambda = [](int a, int b) { return a + b; };
    int result = lambda(3, 4);
    std::cout << "Result: " << result << std::endl;
    return 0;
}
```

编译器会将上述 Lambda 表达式转换为类似下面的代码：

```
#include <iostream>
```

```
// 编译器生成的未命名类
```

```
class __lambda_4_13 {
public:
    __lambda_4_13() = default;
    inline int operator()(int a, int b) const {
        return a + b;
    }
};
```

```
int main() {
    __lambda_4_13 lambda;
    int result = lambda(3, 4);
    std::cout << "Result: " << result << std::endl;
    return 0;
}
```

delete 函数和 default 函数的区别是什么？

- delete 函数：= delete 表示该函数不能被调用。
- default 函数：= default 表示编译器生成默认的函数，例如：生成默认的构造函数。

```
#include <iostream>
using namespace std;
```

```
class A
{
```

```

public:
    A() = default; // 表示使用默认的构造函数
    ~A() = default; // 表示使用默认的析构函数
    A(const A &) = delete; // 表示类的对象禁止拷贝构造
    A &operator=(const A &) = delete; // 表示类的对象禁止拷贝赋值
};

int main()
{
    A ex1;
    A ex2 = ex1; // error: use of deleted function 'A::A(const A&) '
    A ex3;
    ex3 = ex1; // error: use of deleted function 'A& A::operator=(const
    return 0;
}

```

C++ 11 nullptr 比 NULL 优势是什么？

两者的区别：

- NULL：预处理变量，是一个宏，它的值是 0，定义在头文件中，即 #define NULL 0。
- nullptr：C++ 11 中的关键字，是一种特殊类型的字面值，可以被转换成任意其他类型。

nullptr 的优势：

- 有类型，类型是 typedef decltype(nullptr) nullptr_t;，使用 nullptr 提高代码的健壮性。
- 函数重载：因为 NULL 本质上是 0，在函数调用过程中，若出现函数重载并且传递的实参是 NULL，可能会出现，不知和哪一个函数匹配的情况；但是传递实参 nullptr 就不会出现这种情况。

```

#include <iostream>
#include <cstring>
using namespace std;

void fun(char const *p) {
    cout << "fun(char const *p)" << endl;
}

void fun(int tmp) {
    cout << "fun(int tmp)" << endl;
}

```

```
int main() {
    fun(nullptr); // fun(char const *p)
    /*
    fun(NULL); // error: call of overloaded 'fun(NULL)' is ambiguous
    */
    return 0;
}
```

C++问题排查

linux程序崩溃怎么定位问题？

如果开启了core dump文件的转存，那么程序崩溃之后，就会生成 core dump 文件，这里面会有程序运行时的堆栈信息，然后我们可以用 gdp 工具去分析 core dump 文件。

```
gdb myapp core
```

java

在 GDB 中，使用 `bt`（backtrace）命令可以查看程序崩溃时的调用栈：

```
(gdb) bt
```

java

如果程序崩溃问题可以重现，页可以使用 GDB 直接调试程序。在编译程序时，使用 `-g` 选项添加调试信息。例如：

```
gcc -g -o myapp myapp.c
```

java

使用 GDB 启动程序并运行，当程序崩溃时，GDB 会停在崩溃的位置。

```
gdb myapp
(gdb) run
```

java

程序崩溃后，使用 `bt` 命令查看调用栈，找出问题所在的函数和代码行。

有些程序会在崩溃时输出错误信息到标准错误输出（stderr），那么这时候直接去看错误日志就行了。

内存泄露怎么避免？

可以使用智能指针，C++提供了智能指针（如std::shared_ptr、std::weak_ptr等），可以自动管理动态分配的内存。智能指针利用了RAII（资源获取即初始化）的原则，在对象生命周期结束时自动释放内存，避免了显式调用delete的繁琐和遗忘。

也可以使用内存泄漏检测工具（如Valgrind等）来分析程序，在程序运行过程中检测内存泄漏，并及时修复。

如果遇到内存泄漏这种问题，你一般是怎么去解决？

打断点定位然后做处理，后来思考对方应该是想让我回答这种处理措施 

- 在程序中加入必要的错误处理代码，避免程序因为异常情况而导致内存泄漏。
- 使用智能指针等RAII机制，自动管理内存，避免手动管理内存的麻烦和出错风险。
- 使用内存分析工具，检测程序中的内存泄漏，并进行相应的修复。

内存泄露怎么检测？

可以用 Valgrind 工具。

首先看一段 C 程序示例，比如：

```
#include <stdlib.h>
int main()
{
    int *array = malloc(sizeof(int));
    return 0;
}
```

编译程序： gcc -g -o main main.c ，比哪一需要加上 -g 选项打开调试，使用 IDE 的可以用 Debug 模式编译。

使用 Valgrind 检测内存使用情况：

```
valgrind --tool=memcheck --leak-check=full ./main
```

结果：

```
==31416== Memcheck, a memory error detector
==31416== Copyright (C) 2002-2017, and GNU GPL'd, by Julian Seward et al
==31416== Using Valgrind-3.13.0 and LibVEX; rerun with -h for copyright
==31416== Command: ./main_c
==31416==
==31416==
==31416== HEAP SUMMARY:
==31416==     in use at exit: 4 bytes in 1 blocks
==31416==   total heap usage: 1 allocs, 0 frees, 4 bytes allocated
==31416==
==31416== 4 bytes in 1 blocks are definitely lost in loss record 1 of 1
==31416==    at 0x4C2DBF6: malloc (vg_replace_malloc.c:299)
==31416==    by 0x400537: main (main.c:5)
==31416==
==31416== LEAK SUMMARY:
==31416==    definitely lost: 4 bytes in 1 blocks
==31416==    indirectly lost: 0 bytes in 0 blocks
==31416==    possibly lost: 0 bytes in 0 blocks
==31416==    still reachable: 0 bytes in 0 blocks
==31416==    suppressed: 0 bytes in 0 blocks
==31416==
==31416== For counts of detected and suppressed errors, rerun with: -v
==31416== ERROR SUMMARY: 1 errors from 1 contexts (suppressed: 0 from 0)
```

先看看输出信息中的 `HEAP SUMMARY`，它表示程序在堆上分配内存的情况，其中的 `1 allocs` 表示程序分配了 1 次内存，`0 frees` 表示程序释放了 0 次内存，`4 bytes allocated` 表示分配了 4 个字节的内存。

另外，Valgrind 也会报告程序是在哪个位置发生内存泄漏。例如，从下面的信息可以看到，程序发生了一次内存泄漏，位置是 `main.c` 文件的第 5 行：

```
==31416== 4 bytes in 1 blocks are definitely lost in loss record 1 of 1
==31416==    at 0x4C2DBF6: malloc (vg_replace_malloc.c:299)
==31416==    by 0x400537: main (main.c:5)
```

C++代码异常core dump会生成一个栈，里面内容是什么？

栈内容就是程序崩溃前的“现场录像”，能帮你顺着调用链找到哪里出了问题，以及当时的变量状态。当时的变量状态。主要包含这些信息：

- **函数调用链**：从崩溃的那个函数开始，一层层往上列所有调用它的函数。比如 A 调用 B，B 调用 C，C 崩溃了，栈里就会显示 C → B → A 的顺序，能看出“崩溃是从哪个入口一步步走到这里的”。
- **每个函数的地址**：记录每个函数在内存中的具体地址（比如 0x400520 ），结合编译时的符号表（如果没被 strip 掉），就能对应到具体是哪个函数（比如 main() 、 func() ）。
- **函数的参数和局部变量**：每个函数调用时传入的参数值、以及函数内部定义的临时变量（比如 int a=5 中的 a ），会按栈的顺序保存。这能帮你排查“是不是参数传错了”“变量值是不是异常了”。
- **栈指针位置**：记录崩溃时栈顶的位置，告诉你当前栈用到了哪里，有没有溢出（比如栈被写满了）。

最新的图解文章都在公众号首发，别忘记关注哦！！如果你想加入百人技术交流群，扫码下方二维码回复「加群」。



扫一扫，关注「小林coding」公众号

图解计算机基础
认准**小林coding**

每一张图都包含小林的认真
只为帮助大家能更好的理解

- ① 关注公众号回复「**图解**」
获取图解系列 PDF
- ② 关注公众号回复「**加群**」
拉你进百人技术交流群

上次更新:

评论

Powered by [GitHub](#) & [Vssue](#)



登录后才能发表评论 | 支持 Markdown 语法

使用 [GitHub](#) 帐号登录后发表评论

使用 [GitHub](#) 登录



评论加载失败